

Computer Systems (SCS 1305 / IS 1202)

Short Note

Contents

1	History of Computer Systems	4
1.1	Generations of Computers	4
1.2	Moore's Law	5
1.3	The Computer Level Hierarchy	5
1.4	The Stored-Program Concept and von Neumann Architecture	6
2	Data Representation	7
2.1	Quantitative and Qualitative Data	7
2.2	Bits, Bytes and Words	7
2.3	Number Systems	7
2.3.1	Decimal $\leftrightarrow$ Binary	7
2.3.2	Hexadecimal Notation and Binary $\leftrightarrow$ Hex	8
2.4	Character Representation	8
2.4.1	ASCII	8
2.4.2	Unicode	9
3	Computer Arithmetic	9
3.1	Binary Addition, Subtraction, Multiplication, Division	9
3.2	Representing Numbers: Problems and Approaches	10
3.2.1	Unsigned Binary Numbers	10
3.2.2	Signed-Magnitude Representation	11
3.2.3	One's Complement	11
3.2.4	Two's Complement	11
3.2.5	Summary of Integer Ranges (8-bit example)	12
3.3	Non-Integer (Fractional) Numbers	13
3.3.1	Fractions in Decimal and in Binary	13
3.3.2	Converting a Decimal Fraction to Binary	13
3.4	Scientific Notation	13
3.5	IEEE 754 Binary Floating-Point Representation	13
3.5.1	32-bit Format	13
3.5.2	Normalization	14
3.5.3	Worked Example: 32-bit Representation of 263.3	14
3.5.4	Special Values, Overflow and Underflow	14
3.5.5	Single vs Double Precision	15
3.6	Limitations of Floating-Point and Rounding Error	15
4	Boolean Algebra and Logic Operators	16
4.1	Basics of Boolean Algebra	16
4.2	Logic Gates	16

4.2.1	AND Gate	16
4.2.2	OR Gate	17
4.2.3	NOT Gate	17
4.2.4	XOR Gate	17
4.2.5	NAND Gate	17
4.2.6	NOR Gate	17
4.2.7	XNOR Gate	17
4.2.8	Buffer	18
4.2.9	Multiple-Input Gates	18
4.2.10	Operator Precedence	18
4.3	Drawing Logic Circuits from Expressions	18
4.4	Truth Tables and Boolean Expressions	18
4.4.1	From an Expression to a Truth Table	18
4.4.2	Canonical Forms: Minterms and Maxterms	18
4.4.3	Converting a Truth Table to Sum of Minterms	19
4.4.4	Converting a Truth Table to Product of Maxterms	19
4.4.5	De Morgan's Theorem	19
4.4.6	Converting a Canonical Form Back to a Truth Table	19
4.5	Boolean Algebraic Laws and Theorems	20
4.6	Simplifying Boolean Expressions Algebraically	20
4.7	Karnaugh Maps (K-Maps)	20
4.7.1	K-Map Grouping Rules	21
4.7.2	Don't Care Conditions	21
4.7.3	Sum of Products vs Product of Sums from a K-Map	21
4.8	Functional Completeness and Universal Gates	21
5	Logic Circuits	22
5.1	Combinational Circuit Design	22
5.1.1	Adders	22
5.1.2	Subtractors	23
5.1.3	Multipliers	24
5.1.4	Decoders	25
5.1.5	Encoders	25
5.1.6	Multiplexers (MUX)	26
5.2	Sequential Logic Circuits	27
5.2.1	Synchronous vs Asynchronous Sequential Circuits	27
5.2.2	Latches	28
5.2.3	Flip-Flops	29
5.2.4	Registers and Counters	31
6	The CPU and Its Organization	31
6.1	Key Components	31
6.2	Main Memory and Addressing	32
6.3	Buses	32
6.4	Clock and Clock Speed	33
6.5	CPU Performance	33
6.6	Von Neumann Architecture	34
6.7	The Fetch-Decode-Execute Cycle	35
6.8	Instruction Set Architecture (ISA)	35
7	Addressing Modes and Instruction-Level Pipelining	35
7.1	Addressing Modes	35

7.1.1	Effective Address and Physical Address	36
7.1.2	Immediate Addressing	36
7.1.3	Direct Addressing	36
7.1.4	Indirect Addressing	36
7.1.5	Register Addressing (Register Direct)	37
7.1.6	Register Indirect Addressing	37
7.1.7	Displacement Addressing	37
7.1.8	Stack Addressing	37
7.1.9	Worked Comparison Example	37
7.2	Instruction-Level Pipelining	38
7.2.1	Pipelining Efficiency and Speedup	38
7.2.2	Advantages, Disadvantages, Limitations and Issues	39
8	Memory Organization	39
8.1	Primary and Secondary Memory	39
8.2	Types of Memory	39
8.3	Flash Memory	40
8.4	Solid State Devices (SSD)	40
8.5	Memory Hierarchy	40
8.6	Typical Data Access Pattern	41
8.7	The CPU–Memory Gap	41
8.8	Principle of Locality	41
8.9	Cache Memory	41
8.9.1	Cache Hierarchy	41
8.10	Cache Performance	42
8.11	Key Design Decisions of a Cache	42
8.11.1	Write Policy	42
8.11.2	Mapping Schemes – Why They Are Needed	42
8.11.3	Cache Lines (Cache Blocks) and Cache Entries	43
8.12	Direct Mapped Cache	43
8.12.1	Address Breakdown: Tag, Index, Offset	43
8.12.2	General Formulas for Tag / Index / Offset Lengths	44
8.12.3	Advantages and Disadvantages of Direct Mapped Cache	45
8.13	Fully Associative Cache	45
8.14	Set Associative Cache	45
8.15	Cache Replacement Policies	46

1 History of Computer Systems

1.1 Generations of Computers

The evolution of computing machinery is classified into generations based on the underlying technology of the era. Each generation brought computers that were smaller, cheaper, faster and more powerful than the previous one. Generations are usually compared in terms of the hardware and software technology used, computing speed (instructions executed per second), physical size, and typical applications.

- **Generation Zero (1642 – 1940): Mechanical Calculating Machines**

- Calculating Clock – Wilhelm Schickard (1623).
- Pascaline (adding machine) – Blaise Pascal (1642–1644).
- Difference Engine – an automatic mechanical calculator designed to tabulate polynomial functions, by Charles Babbage (1819–1822). Babbage also designed the Analytical Engine, a Victorian-era general purpose computer (1880s).
- The **Atanasoff–Berry Computer (ABC)**, built by John Vincent Atanasoff and Clifford Berry at Iowa State University (1937–1942), is regarded as the world’s first electronic digital computer. It was the size of a large desk, weighed about 750 pounds, used rotating drums for memory, vacuum tubes, and recorded numbers by scorching marks on cards.
- **Patent controversy:** J. Presper Eckert and John Mauchly (developers of ENIAC) were originally credited with the first patent for an electronic digital computer. In 1973 a US District Judge overturned the ENIAC patent, ruling that Eckert and Mauchly had derived their ideas from Atanasoff’s work.

- **First Generation (1940 – 1956)**

- Used **vacuum tubes** for circuitry and magnetic drums for memory.
- Enormous machines, often occupying entire rooms.
- Programmed directly in machine language.
- Expensive to run, consumed large amounts of electricity, generated heat that frequently caused malfunctions.
- Examples: **ENIAC** (Electronic Numerical Integrator and Computer, 1946) – the first general-purpose electronic digital computer; **UNIVAC** (Universal Automatic Computer, 1951) – the first commercially produced electronic digital computer; **IBM 650** – the first mass-produced computer.
- Disadvantages: bulky, vacuum tubes burned out frequently, produced a lot of heat, high maintenance.

- **Second Generation (1956 – 1963)**

- **Transistors** replaced vacuum tubes.
- Programming moved from cryptic binary machine code to symbolic (assembly) languages; early high-level languages such as COBOL and FORTRAN emerged.
- These were the first computers to store instructions in memory.
- Advantages: much smaller in size, faster, more reliable.
- Disadvantages: still overheated quickly, maintenance problems remained.

- **Third Generation (1964 – 1971)**

- Hallmark: development of the **integrated circuit (IC)**. Transistors were miniaturised and placed on silicon chips (semiconductors).
- Users began interacting with computers through keyboards and monitors, with an operating system managing the machine, allowing several applications to run at once.
- Examples: IBM 360, DEC PDP-8/PDP-11, Cray-1 supercomputer.
- Advantages: ICs are very small, improved performance, cheaper production.
- Disadvantage: ICs are sophisticated to design and fabricate.

- **Fourth Generation (1971 – present)**

- Based on **Very Large Scale Integration (VLSI)**, packing more than 10,000 components onto a single chip.
- The **microprocessor** (e.g. the Intel 4004, 1971) placed the CPU, memory and I/O control circuitry onto a single chip.
- Saw the development of GUIs, the mouse, and handheld devices.

- **Fifth Generation (present and beyond)**

- Based on **artificial intelligence** and **ULSI** (Ultra Large Scale Integration) technology.
- Still evolving; some applications already exist, e.g. voice recognition.
- Uses parallel processing and superconductors to move towards true AI.
- Goal: devices that understand natural language and are capable of learning and self-organisation.

- **“Sixth Generation” (emerging, not officially numbered)**

- Associated with nanotechnology, multi-core / multi-CPU processing, and quantum computing (qubits), which can in principle perform certain calculations far faster than conventional bits.
- Note: this numbering of computer *generations* is not the same thing as, for example, “Intel generations” of processors – the two should not be confused.

1.2 Moore’s Law

In 1965, Gordon Moore observed that the number of transistors that could be placed on a chip was doubling roughly every 18–24 months. This observation became known as **Moore’s Law**: “the number of transistors on a chip approximately doubles every two years.” Although called a law, it is really an industry trend/self-fulfilling prediction rather than a physical law of nature, and it cannot continue indefinitely because of physical limits on how small transistors can be made.

1.3 The Computer Level Hierarchy

A computer system can be understood as a stack of abstraction layers. Each layer (level) is an abstraction built on top of the layer below it: machines at a given level execute their own instructions and call on the machines/services at lower levels to carry out the actual work. Ultimately, everything is carried out by digital circuits.

Level	Description
Level 6: User Level	Program execution and the user interface; the level we interact with most directly.
Level 5: High-Level Language Level	The level at which we write programs in languages such as C, Pascal, Java, etc.
Level 4: Assembly Language Level	Works on assembly code produced from Level 5 (or written directly at this level).
Level 3: System Software Level	Controls the execution of processes on the system; assembly instructions often pass through this level unmodified.
Level 2: Machine Level (ISA Level)	Also called the Instruction Set Architecture level. Consists of instructions specific to the machine's architecture. Programs at this level need no compiler/interpreter/assembler.
Level 1: Control Level	A control unit decodes and executes machine instructions and moves data through the system. Hardwired control units execute machine instructions directly with hardware.
Level 0: Digital Logic Level	Digital circuits (chips) made of gates and wires; these components physically implement the logic of every level above.

A single high level statement, e.g. $z = x * y;$, is compiled down through assembly language and eventually into binary machine code that the hardware executes – illustrating how a program descends through these levels from Level 5 to Level 0.

1.4 The Stored-Program Concept and von Neumann Architecture

On the ENIAC, all programming was done at the digital logic level – literally by moving plugs and rewiring the machine. A different hardware configuration was required for every new problem, so even a “simple” calculation could take days of skilled labour to set up.

The idea of **stored-program computers** – storing the program in memory alongside data – was proposed by the mathematician **John von Neumann** in 1945, in the paper “First Draft of a Report on the EDVAC”. Such computers are called **von Neumann Architecture** machines.

Key characteristics of stored-program (von Neumann) computers:

- Three principal hardware systems: a Central Processing Unit (CPU), a main memory system, and an I/O system.
- The capacity to carry out sequential instruction processing.
- A single data path between the CPU and main memory, known as the **von Neumann bottleneck**, since only one word can travel along it at a time.
- Instructions and data share the same memory space and are fetched/executed using a **fetch–decode–execute cycle**.

Because a single shared data path limits performance, enormous gains in computing power have required designs that depart from the classic von Neumann model (e.g. multiple buses, caches, pipelining), even though most modern computers remain fundamentally von Neumann machines. (A detailed diagram and discussion of von Neumann Architecture is given in Section 6.6.)

2 Data Representation

2.1 Quantitative and Qualitative Data

Data representation is concerned with how numbers, characters and symbols are represented inside a computer.

- **Quantitative data** can be counted or measured and expressed using numbers. It may be integer or non-integer, and signed or unsigned.
- **Qualitative data** is descriptive/conceptual, and is categorised based on traits or characteristics rather than numeric value.

2.2 Bits, Bytes and Words

- A **bit** (binary digit) is the smallest unit of data in a computer; it holds either 0 or 1.
- A **byte** is 8 consecutive bits, capable of storing a single character.
- A **word** is the natural, fixed-size unit of data handled as a whole by a particular processor's instruction set / hardware. The number of bits in a word (word size / word length) is a defining characteristic of a processor architecture.

Storage hierarchy (all base 1024, i.e. 2^{10}):

8 bits	= 1 Byte
1024 Bytes	= 1 Kilobyte (KB)
1024 KB	= 1 Megabyte (MB)
1024 MB	= 1 Gigabyte (GB)

2.3 Number Systems

System	Alphabet (allowed digits)
Decimal (base 10)	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
Octal (base 8)	{0, 1, 2, 3, 4, 5, 6, 7}
Hexadecimal (base 16)	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}
Binary (base 2)	{0, 1}

A **leading zero** is any 0 digit that appears before the first non-zero digit in a positional number; leading zeros are not significant – they are simply placeholders (e.g. the “007” in James Bond conveys nothing extra numerically).

2.3.1 Decimal $\leftrightarrow$ Binary

Decimal to binary: repeatedly divide the decimal number by 2, recording the remainder at each step, until the quotient is 0. The binary result is the sequence of remainders read from *last to first* (the last non-zero quotient's remainder is the most significant bit).

Example: Convert $(123)_{10}$ to binary.

$$\begin{aligned} 123 \div 2 &= 61 \text{ r } 1, & 61 \div 2 &= 30 \text{ r } 1, & 30 \div 2 &= 15 \text{ r } 0, & 15 \div 2 &= 7 \text{ r } 1, \\ & & 7 \div 2 &= 3 \text{ r } 1, & 3 \div 2 &= 1 \text{ r } 1, & 1 \div 2 &= 0 \text{ r } 1 \end{aligned}$$

Reading the remainders from bottom (last) to top (first) gives $(123)_{10} = (1111011)_2$.

Binary to decimal: multiply each bit by 2 raised to the power of its position (position 0 is the least significant / rightmost bit), and sum the results.

Example: Convert 10011010_2 to decimal.

Bit position	7	6	5	4	3	2	1	0
Bit value	1	0	0	1	1	0	1	0
Place value	128	64	32	16	8	4	2	1
$128 + 16 + 8 + 2 = 154 \Rightarrow 10011010_2 = 154_{10}$								

A useful alternative technique for binary to decimal conversion: work from the most significant bit, and at each position ask whether that power of two “fits” into what remains of the number, placing a 1 or 0 accordingly and subtracting if it fits, then continuing to the next lower power of two (this is effectively the same process worked in reverse, useful for converting decimal into binary directly by comparison against powers of two).

2.3.2 Hexadecimal Notation and Binary $\leftrightarrow$ Hex

Each hex digit corresponds to exactly 4 bits (a “nibble”):

HEX	Bit Pattern	HEX	Bit Pattern
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Binary to hexadecimal: group the binary digits into sets of 4 starting from the least significant bit (pad with leading zeros on the left if necessary), then convert each group of 4 bits to its hex digit.

Examples:

$$10010110_2 \Rightarrow 1001 \ 0110 \Rightarrow 9 \ 6 \Rightarrow 96_{16}$$

$$11011011_2 \Rightarrow 1101 \ 1011 \Rightarrow D \ B \Rightarrow DB_{16}$$

$$0010100111110101_2 \Rightarrow 0010 \ 1001 \ 1111 \ 0101 \Rightarrow 2 \ 9 \ F \ 5 \Rightarrow 29F5_{16}$$

Hexadecimal to binary: reverse the process – expand each hex digit into its 4-bit binary pattern and concatenate.

Decimal $\leftrightarrow$ hexadecimal / octal: convert via binary (decimal $\rightarrow$ binary $\rightarrow$ hex/octal), or repeatedly divide by 16 (hex) / 8 (octal) and read the remainders from last to first, analogous to the decimal-to-binary method.

Binary $\leftrightarrow$ octal: group bits into sets of 3 (each octal digit corresponds to 3 bits), starting from the least significant bit, and convert each group.

Note: the reason grouping works is that $16 = 2^4$ and $8 = 2^3$, so each hex/octal digit exactly represents a fixed-width group of binary digits.

2.4 Character Representation

2.4.1 ASCII

ASCII (American Standard Code for Information Interchange) uses 7-bit patterns to represent:

- Letters of the English alphabet: a--z and A--Z
- Digits 0--9
- Punctuation symbols: () [] { } ' " ! / \
- Arithmetic operator symbols: + - * < > =
- Special symbols: space, % \$ # & @ ^

With 7 bits, $2^7 = 128$ characters can be represented; the 8th bit was historically used as a **parity bit** for simple error checking. As reliability of hardware improved, this need for a parity bit faded, and manufacturers extended ASCII using the freed 8th bit to represent an additional 128 special characters (codes 128 to 255, i.e. 2^7 to $2^8 - 1$) – this is known as **Extended ASCII**.

Uppercase and lowercase versions of the same letter differ by exactly $0x20$ (e.g. A = $0x41$, a = $0x61$).

2.4.2 Unicode

ASCII and EBCDIC are built around the Latin alphabet and cannot represent non-Latin scripts well, so many countries historically developed their own incompatible codes for native languages.

Unicode solves this by using a much larger code space:

- Originally a 16-bit system, giving $2^{16} = 65,536$ possible characters, and is **downward compatible** with ASCII and Latin-1.
- As of the standard's growth (e.g. Unicode 13.0, March 2020), it defines over 140,000 characters covering 150+ modern and historic scripts plus symbol sets and emoji.
- Unicode version 3.0 (September 1999) added support for **Sinhala**.
- Characters are grouped into **blocks**; e.g. the Sinhala block occupies code page 0D80 to 0DFF (128 code points).
- The Unicode Consortium was incorporated in California in January 1991.
- Unicode supports three encoding forms that all use the same character repertoire: **UTF-8**, **UTF-16** and **UTF-32**.

Example: the ASCII code for 'A' is $0x41$; the Unicode code point for 'A' is written U+0041 (i.e. 00 41 in the Basic Latin block), showing the downward compatibility with ASCII.

3 Computer Arithmetic

3.1 Binary Addition, Subtraction, Multiplication, Division

Binary addition rules:

$$0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 0 = 1, \quad 1 + 1 = 10 \text{ (write 0, carry 1)}$$

Example:

$$\begin{array}{r} 11111 \\ 01101 \\ + 10111 \\ \hline 100100 \end{array}$$

Binary subtraction rules:

$$0 - 0 = 0, \quad 0 - 1 = 1 \text{ (with borrow)}, \quad 1 - 0 = 1, \quad 1 - 1 = 0$$

Example:

$$\begin{array}{r}
 \text{*** (borrows)} \\
 101101 \\
 - 010111 \\
 \hline
 010110
 \end{array}$$

Binary multiplication: follows the same shift-and-add process as decimal long multiplication – the multiplicand is multiplied by each bit of the multiplier in turn, each partial product is shifted one place left, and the partial products are summed.

$$\begin{array}{r}
 1011 \text{ (multiplicand)} \\
 \times 1010 \text{ (multiplier)} \\
 \hline
 0000 \\
 1011 \\
 0000 \\
 1011 \\
 \hline
 1101110
 \end{array}$$

Binary division: follows the same long-division process as decimal division, except at each step the quotient digit can only be 0 or 1.

3.2 Representing Numbers: Problems and Approaches

Any scheme for representing numbers in binary must deal with:

- How to represent both positive and negative numbers.
- Where the radix (binary) point goes, for non-integers.
- The range of values that can be represented in a fixed number of bits.

Three broad approaches exist:

- **Unsigned representation** – for non-negative integers only.
- **Signed representation** – for integers (positive and negative).
- **Floating-point representation** – for numbers with fractional parts.

Format for an n -bit unsigned number: all n bits, $b_{n-1} \dots b_1 b_0$, form the magnitude, with b_{n-1} as the most significant bit (MSB).

Format for an n -bit signed number: bit b_{n-1} is the sign bit (0 = positive, 1 = negative) and the remaining $n - 1$ bits, $b_{n-2} \dots b_1 b_0$, form the magnitude.

3.2.1 Unsigned Binary Numbers

- Only positive numbers (including zero) can be stored.
- Smallest 8-bit value: $00000000 = 0$.
- Largest 8-bit value: $11111111 = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255 = 2^8 - 1$.
- General range for n bits: 0 to $2^n - 1$ (i.e. 2^n distinct values).

3.2.2 Signed-Magnitude Representation

- The leftmost (most significant) bit is a dedicated **sign bit**: 0 = positive, 1 = negative. The remaining bits store the magnitude, unchanged, as for unsigned numbers.
- Smallest positive 8-bit value: $00000000 = 0$.
- Largest positive 8-bit value: $01111111 = 64 + 32 + 16 + 8 + 4 + 2 + 1 = 127 = 2^7 - 1$.
- General range for positive numbers, n bits: 0 to $2^{n-1} - 1$.
- **Problem 1 – two representations of zero**: 00000000 (+0) and 10000000 (−0) both represent zero, which is ambiguous.
- **Problem 2 – arithmetic is awkward**: both sign and magnitude must be considered explicitly when adding/subtracting; naive bit-wise addition of the sign bits along with magnitude bits gives wrong answers. *Example*: $5 + (-3)$ using 8-bit sign-magnitude: $00000101 + 10000011 = 10001000$, which if read again as sign-magnitude means -8 – clearly wrong (the correct answer is 2). This shows that simple bit-wise addition does not work correctly for sign-magnitude numbers.

3.2.3 One's Complement

- Positive numbers are unchanged from their unsigned form (and still start with a 0).
- A negative number is represented by **inverting every bit** (0s become 1s and 1s become 0s) of the corresponding positive number – e.g. one's complement of $(10010100)_2$ is $(01101011)_2$ is not quite the right pairing; more precisely, to get $-x$, take the positive representation of x and flip every bit.
- Since positive numbers always start with 0, their complements (the negative numbers) always start with 1.
- For an n -bit one's complement number, the representable range is $-(2^{n-1} - 1) \leq D \leq +(2^{n-1} - 1)$, e.g. a 4-bit one's complement number ranges from -7 to $+7$ (with two representations of zero).
- **Subtraction using one's complement** requires an **end-around carry**: if the addition produces a carry out of the most significant bit (a 9th bit for an 8-bit system), that carry bit is added back into the least significant bit of the result. *Example*: $5 + (-3)$ in 8-bit one's complement:

$$00000101 + 11111100 = 1\,00000001 \text{ (9-bit result, carry = 1)}$$

$$00000001 + 1 \text{ (carry)} = 00000010 = +2 \quad \checkmark$$

- **Benefits**: easy to negate (just flip all bits); subtraction can be performed as addition.
- **Problems**: still has two representations of zero (00000000 and 11111111); requires the extra end-around-carry step, which is not “clean”.

3.2.4 Two's Complement

- Positive numbers are unchanged from their unsigned form.
- A negative number is obtained by taking the one's complement (bit-flip) of the corresponding positive number and then **adding 1**. In short, **two's complement = one's complement + 1**.

- Equivalent rule: starting from the signed binary representation of the positive value, copy bits from right to left unchanged until (and including) the first 1 is copied; then complement (flip) all remaining bits to the left. (An important exception: 10000000 in 8 bits represents -128 , which has no positive counterpart in the same width.)
- For an n -bit two's complement number, the representable range is $-(2^{n-1}) \leq D \leq +(2^{n-1} - 1)$ – e.g. for 8 bits: -128 to $+127$.
- **Subtraction using two's complement:** simply add the two's complement of the number being subtracted; any carry out of the most significant bit (the 9th bit, for 8-bit numbers) is **simply discarded** (no end-around carry needed). *Example:* $5 + (-3)$ in 8-bit two's complement:

$$00000101 + 11111101 = 1\,00000010 \Rightarrow \text{discard overflow bit} \Rightarrow 00000010 = +2 \quad \checkmark$$

- **Benefits of two's complement:** only one representation of zero; arithmetic (addition and subtraction) works uniformly using ordinary binary addition; subtraction $A - B$ can always be computed as $A + (\text{two's complement of } B)$.
- **Drawback:** the range is asymmetric (e.g. for 4 bits: $+7$ to -8), and computing the negative of a number requires bit inversion plus an increment (a small amount of extra work compared to just flipping the sign bit).

8-bit two's complement reference table (selected values):

Bit pattern	Value
01111111	+127
$\vdots$	$\vdots$
00000011	+3
00000010	+2
00000001	+1
00000000	0
11111111	-1
11111110	-2
11111101	-3
$\vdots$	$\vdots$
10000001	-127
10000000	-128

3.2.5 Summary of Integer Ranges (8-bit example)

Representation	Largest	Smallest
8-bit unsigned	255	0
8-bit two's complement	127	-128

When subtracting two numbers $a - b$, instead of designing a separate subtraction circuit, it is more practical to **negate b (two's complement) and then add**, since addition hardware (and two's complement negation) is already available – this is exactly why two's complement is the standard representation used by real computer hardware.

3.3 Non-Integer (Fractional) Numbers

3.3.1 Fractions in Decimal and in Binary

A number with a fractional part can be expressed as a sum of place values with negative exponents.

Decimal example:

$$16.357 = 1 \times 10^1 + 6 \times 10^0 + 3 \times 10^{-1} + 5 \times 10^{-2} + 7 \times 10^{-3}$$

Binary example: 10.011_2

$$10.011_2 = 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} = 2 + \frac{1}{4} + \frac{1}{8} = 2\frac{3}{8}_{10}$$

3.3.2 Converting a Decimal Fraction to Binary

Convert the integer part normally (repeated division by 2). For the fractional part, repeatedly **multiply by 2**; each time, the digit to the left of the new decimal point (0 or 1) is the next binary fraction digit, and the process continues with the remaining fractional part until it becomes 0 (or the desired precision is reached).

Example: Convert 16.375_{10} to binary.

- Integer part: $16_{10} = 10000_2$.
- Fractional part 0.375:

$$0.375 \times 2 = 0.750 \Rightarrow 0, \quad 0.750 \times 2 = 1.500 \Rightarrow 1, \quad 0.500 \times 2 = 1.000 \Rightarrow 1$$

Reading the generated digits from first to last: $0.375_{10} = 0.011_2$.

Therefore $16.375_{10} = 10000.011_2$.

3.4 Scientific Notation

Decimal scientific notation:

$$135.26 = 1.3526 \times 10^2, \quad 13,526,000 = 1.3526 \times 10^7, \quad 0.0000002452 = 2.452 \times 10^{-7}$$

In 1.3526×10^7 : the **mantissa** (significand) is 3526, the **base** is 10, and the **exponent** is 7.

Binary scientific notation: a binary fraction can similarly be normalised, e.g.

$$0.000001001_2 = 1.001_2 \times 2^{-6}$$

by shifting the radix point so that exactly one non-zero digit remains to its left, and adjusting the exponent accordingly.

3.5 IEEE 754 Binary Floating-Point Representation

3.5.1 32-bit Format

A single-precision (32-bit) floating-point number is laid out as:

$$\underbrace{1 \text{ bit}}_{\text{Sign}} \quad \underbrace{8 \text{ bits}}_{\text{Biased Exponent}} \quad \underbrace{23 \text{ bits}}_{\text{Significand (Mantissa)}}$$

- **Sign bit:** 0 = positive, 1 = negative.

- **Biased exponent** (8 bits): rather than using two's complement or a separate sign for the exponent, a fixed **bias** is *added* to the true exponent before storing it. For 32-bit format the bias is 127 ($= 2^{8-1} - 1$); in general the bias equals $2^{k-1} - 1$ where k is the number of exponent bits.

$$\text{Biased Exponent} = \text{True Exponent} + 127$$

With an 8-bit field, the *stored* biased exponent value ranges over 0 to 255, but the two extreme patterns (00000000 and 11111111) are reserved for special values (see below), so the usable **true** exponent range is -126 to $+127$.

- **Significand / Mantissa** (23 bits): stores the fractional part of a *normalized* number.

3.5.2 Normalization

A number is **normal** (normalized) if the most significant digit of its significand is non-zero. In binary, a normal non-zero number has the form

$$\pm 1.f_1f_2f_3 \dots \times 2^e$$

Because the leading bit of a normalized binary significand is *always* 1, it does not need to be stored explicitly – it is an **implicit bit**, which effectively gives 24 bits of precision from only 23 stored bits.

- **Explicit normalization:** move the radix point to the *left* of the MSB (used when the number looks like 0.00110_2 , giving e.g. 0.110×2^{-2}).
- **Implicit normalization:** move the radix point to the *right* of the MSB (used when the number looks like 101.100_2 , giving e.g. 1.01100×2^2).

3.5.3 Worked Example: 32-bit Representation of 263.3

$$\begin{aligned} 263_{10} &= 100000111_2 \\ 0.3_{10} &\approx 0.0100110011001101_2 \\ 263.3_{10} &\approx 100000111.0100110011001101_2 \\ &= 1.000001110100110011001101 \times 2^8 \quad (\text{normalized}) \end{aligned}$$

- Sign bit = 0 (positive number).
- True exponent = 8 $\Rightarrow$ Biased Exponent = $8 + 127 = 135 = 10000111_2$.
- Mantissa (drop the implicit leading 1, keep 23 bits, truncating/rounding the LSB as needed): 00000111010011001100110.

Full 32-bit pattern:

$$0 \ 10000111 \ 00000111010011001100110$$

3.5.4 Special Values, Overflow and Underflow

- An all-zero biased exponent (-127 true, all-0 field) together with an all-zero significand represents ± 0 ; more generally, an all-zero exponent field with a non-zero significand represents a **denormalized (subnormal)** number, allowing representation of numbers smaller than the smallest normal number, at the cost of reduced precision.
- An all-one biased exponent ($+128$ true, all-1 field) is reserved: combined with a zero significand it represents $\pm\infty$ (e.g. $1/0 = \infty$); combined with a non-zero significand it represents **NaN** (Not a Number), e.g. $\sqrt{-1}$.

- **Floating-point overflow:** occurs when the true exponent needed exceeds the maximum representable value (127 for 32-bit format); the result effectively becomes $\pm\infty$. *Example:* $1.111111 \times 2^{127} + 1.111111 \times 2^{127} = 11.111110 \times 2^{127} \approx 1.111110 \times 2^{128}$, which cannot be represented since $128 > 127$.
- **Floating-point underflow:** occurs when the true exponent needed is smaller than the minimum representable value (-126 for 32-bit format); the result effectively becomes 0. *Example:* $10.010000 \times 2^{-126} \div 11.000000 \times 2^0 = 0.110000 \times 2^{-126} \approx 1.100000 \times 2^{-127}$, which underflows to approximately 0.
- **Range summary (32-bit):** negative numbers between $-(2 - 2^{-23}) \times 2^{128}$ and -2^{-127} ; positive numbers between 2^{-127} and $(2 - 2^{-23}) \times 2^{128}$. The largest finite magnitude is approximately $(2 - 2^{-23}) \times 2^{127} \approx 3.4028235 \times 10^{38}$.

3.5.5 Single vs Double Precision

Format	Total bits	Exponent bits	Significand bits
Single-precision (<code>float</code> in C/C++)	32	8	23
Double-precision (<code>double</code> in C/C++)	64	11	52

IEEE Standard 754 (adopted 1985, revised 2008) is the most important floating-point standard, developed to make numerical programs portable across processors; it uses this same biased (“excess- N ”) exponent scheme for various precisions (e.g. excess-15 for half precision, excess-127 for single, excess-1023 for double, excess-16383 for quad precision).

3.6 Limitations of Floating-Point and Rounding Error

- Floating-point representation can only **approximate** real numbers; more bits reduce the error but can never eliminate it entirely.
- Overflow and underflow can cause a program to crash, or silently produce erroneous, hard-to-detect results.
- When a smaller floating-point number’s exponent is adjusted to match a larger number before addition, some of its least-significant bits may be lost – this loss of precision is called **rounding**.
- A **round-off (rounding) error** is the difference between the exact mathematical result and the result obtained using finite-precision arithmetic. This is a form of **quantization error**, distinct from a **truncation error** (simply chopping off digits without rounding).
- Two common rounding rules:
 - **Round-by-chop:** if the discarded portion is less than 0.5 (in the units of the last kept digit), the number is simply truncated (chopped).
 - **Round-to-nearest** (used by the IEEE standard): if the discarded portion is greater than 0.5, add 1 to the least significant bit (LSB) of the mantissa, carrying if necessary. If the discarded portion is exactly 0.5, add 1 to the LSB only if the LSB is currently 1 (i.e. round to the nearest *even* value); otherwise leave it unchanged.

Example (6 significant decimal digits): representing $\pi = 3.14159265358979\dots$: round-by-chop gives 3.14159; round-to-nearest gives 3.14159 as well here, but for 3.1415935... chop would give 3.14159 while round-to-nearest would give 3.14160.

- **Worked example – 16-bit float:** represent 1008.8125_{10} using a 16-bit format with 1 sign bit, 5 exponent bits and 10 mantissa bits.

$$1008_{10} = 1111110000_2, \quad 0.8125_{10} = 0.1101_2$$

$$1008.8125_{10} = 1111110000.1101_2 = 1.1111100001101 \times 2^9$$

Biased exponent = $9 + 15 = 24 = 11000_2$ (bias for a 5-bit exponent field is $2^{5-1} - 1 = 15$). Before rounding: 0 11000 1111100001101. Since the mantissa has more than 10 bits available, it must be rounded to 10 bits, giving: 0 11000 1111100010 (after rounding up the final kept bit).

- **Real-world consequence:** the 1991 Patriot missile failure during the Gulf War (Dhahran, Saudi Arabia) was traced to a computer arithmetic rounding error that caused an inaccurate calculation of elapsed time since system boot, which caused the system to fail to intercept an incoming Scud missile.

4 Boolean Algebra and Logic Operators

4.1 Basics of Boolean Algebra

Boolean algebra was introduced by George Boole (a British mathematician) in 1854. It is the branch of algebra in which variables take only two truth values: **true** and **false**, conventionally written as 1 and 0.

The three basic Boolean operations are:

Operation	Symbol(s)	Name
AND	$A \cdot B$ or $A \wedge B$	Conjunction
OR	$A + B$ or $A \vee B$	Disjunction
NOT	$\overline{A}$ or $\neg A$	Negation

A **Boolean function** is an expression built from binary variables combined using Boolean operators, e.g. $F(x, y) = x \cdot \overline{y} + \overline{x} \cdot (x + y)$. Since each variable can only take 2 values, a function of n variables has 2^n possible input combinations, and can be fully described by a **truth table** with 2^n rows.

4.2 Logic Gates

A **logic gate** is a piece of hardware (built internally from transistors, resistors and diodes) that implements a Boolean operation on its inputs. The behavioural diagram of any Boolean expression can be represented as a circuit built from logic gates.

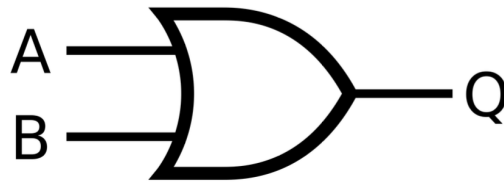
4.2.1 AND Gate

$$Q = A \cdot B$$



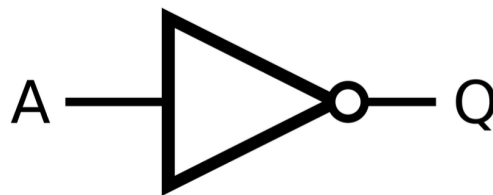
4.2.2 OR Gate

$$Q = A + B$$



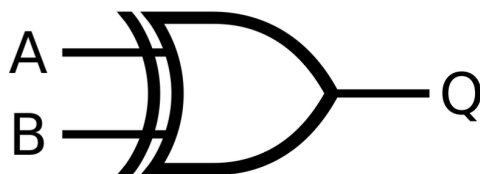
4.2.3 NOT Gate

$$Q = \overline{A}$$



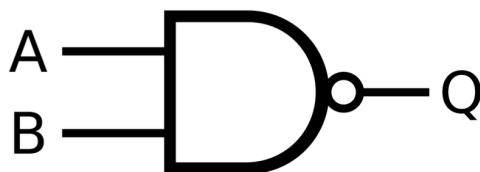
4.2.4 XOR Gate

$$Q = A \oplus B \text{ (output is 1 only when the inputs differ)}$$



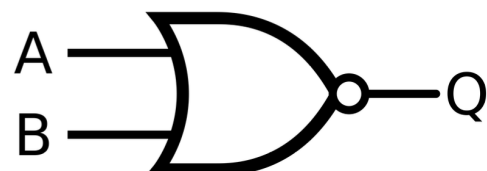
4.2.5 NAND Gate

$$Q = \overline{A \cdot B}$$



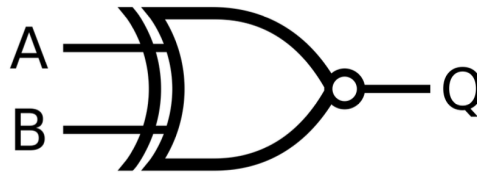
4.2.6 NOR Gate

$$Q = \overline{A + B}$$



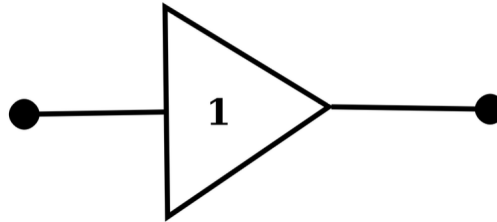
4.2.7 XNOR Gate

$$Q = \overline{A \oplus B} \text{ (output is 1 only when the inputs are equal)}$$



4.2.8 Buffer

A buffer passes its input to its output unchanged ($Q = A$). It is used to increase the propagation delay of a circuit, e.g. to help match timing between paths.



4.2.9 Multiple-Input Gates

Gates with more than two inputs are also available (typically built into ICs, one gate type per IC, sometimes with multiple gates per chip). For example:

$$F = A \wedge B \wedge C \quad (3\text{-input AND}), \quad F = A \vee B \vee C \quad (3\text{-input OR}), \quad F = A \oplus B \oplus C \quad (3\text{-input XOR})$$

4.2.10 Operator Precedence

When evaluating or simplifying Boolean expressions, the operator precedence (highest to lowest) is:

$$() > \text{NOT}(\sim) > \text{AND}(\wedge) > \text{OR}(\vee)$$

4.3 Drawing Logic Circuits from Expressions

Any Boolean expression can be translated directly into a circuit diagram by substituting each operator for its corresponding gate and wiring the variables into the correct gates, respecting operator precedence. More complex expressions require more gates wired together; note that the *same* Boolean function can often be drawn as several different circuit layouts (different gate arrangements) that are logically equivalent but not necessarily equally efficient.

4.4 Truth Tables and Boolean Expressions

4.4.1 From an Expression to a Truth Table

Given a Boolean expression with n variables, there are 2^n possible input combinations, so the truth table will have 2^n rows. Substitute each combination of variable values into the expression to compute the corresponding output.

Example: $F(x, y, z) = x.y.\bar{z} + y.\bar{z}$ has 3 variables, so its truth table has $2^3 = 8$ rows, computed by substituting each of the 8 combinations of x, y, z .

4.4.2 Canonical Forms: Minterms and Maxterms

When converting a truth table into a Boolean expression, there are two standard *canonical* forms, in which every term must contain **all** the variables of the function.

- **Minterm (Standard Product):** a term formed by AND-ing together all the variables of the function, each either in its normal or complemented form. For 3 variables there are $2^3 = 8$ possible minterms (e.g. $\bar{x}\bar{y}\bar{z}$, $\bar{x}\bar{y}z$, ..., $x.y.z$). Note that a term like $(x.y)$ alone is *not* a minterm for a 3-variable function, since it is missing z .
- **Maxterm (Standard Sum):** a term formed by OR-ing together all the variables of the function, each either in its normal or complemented form. For 3 variables there are $2^3 = 8$ possible maxterms (e.g. $\bar{x} + y + z$, ..., $x + y + z$).
- **Sum of Minterms (SoM):** a Boolean function expressed as an OR (disjunction) of minterms – also called **Sum of Products** in canonical form.
- **Product of Maxterms (PoM):** a Boolean function expressed as an AND (conjunction) of maxterms – also called **Product of Sums** in canonical form.

Standard forms (as opposed to canonical forms) allow terms that do *not* need to contain every variable of the function, e.g. $F = \bar{x}.y + x$ is in **Sum of Products** standard form even though the term x alone does not mention y .

4.4.3 Converting a Truth Table to Sum of Minterms

Every row of the truth table where the output is 1 corresponds to exactly one minterm: for that row, write each variable un-complemented if it is 1 in that row, and complemented if it is 0. The overall function is the OR (disjunction) of the minterms for every row where the output is 1 (this works because the expression can only equal 1 when the input matches one of these specific rows – a disjunction of the conditions under which the output is true).

Example: for the rows (indexed from 0) where the output is 1: 2, 4, 5, 6, 7, we would write $F(x, y, z) = \sum m(2, 4, 5, 6, 7)$ using **minterm shorthand notation**, meaning the sum (OR) of minterms m_2, m_4, m_5, m_6, m_7 .

4.4.4 Converting a Truth Table to Product of Maxterms

Symmetrically: every row where the output is 0 corresponds to one maxterm. For that row, write each variable complemented if it is 1 in that row, and un-complemented if it is 0 (i.e. the opposite convention from minterms), then AND all these maxterms together.

This can be justified via **De Morgan's Theorem**: if F' (the complement of F) can be written as a sum of minterms (using the rows where $F = 0$), then $F = \overline{F'}$ can be obtained by applying De Morgan's theorem to convert that sum of (complemented) minterms into a product of maxterms.

Shorthand notation: $F(x, y, z) = \prod M(0, 1, 3)$ means the product (AND) of maxterms M_0, M_1, M_3 (i.e. rows 0, 1 and 3 have output 0).

4.4.5 De Morgan's Theorem

$$\overline{A \cdot B} = \overline{A} + \overline{B}, \quad \overline{A + B} = \overline{A} \cdot \overline{B}$$

This theorem generalises to any number of variables, and is essential for converting between Sum-of-Products and Product-of-Sums forms, and for implementing circuits using a single gate type (see Section 4.8).

4.4.6 Converting a Canonical Form Back to a Truth Table

- **From SoM:** for each minterm in the expression, find the truth table row it corresponds to and fill in a 1 in the output column; fill all remaining rows with 0.

- **From PoM:** for each maxterm, negate every literal in it and AND them together to find the corresponding row (by De Morgan’s theorem, this identifies the row that makes the *original* maxterm equal to 0); fill in a 0 for that row. Fill all remaining rows with 1.

4.5 Boolean Algebraic Laws and Theorems

Law		
Annulment Law	$A \cdot 0 = 0$	$A + 1 = 1$
Identity Law	$A + 0 = A$	$A \cdot 1 = A$
Idempotent Law	$A + A = A$	$A \cdot A = A$
Complement Law	$A + \bar{A} = 1$	$A \cdot \bar{A} = 0$
Commutative Law	$A \cdot B = B \cdot A$	$A + B = B + A$
Double Negation Law	$\bar{\bar{A}} = A$	
Distributive Law	$A(B + C) = A \cdot B + A \cdot C$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Redundancy Law	$A + A \cdot B = A$	$A \cdot (A + B) = A$
De Morgan’s Law	$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \cdot \bar{B}$

There are also two additional useful identities sometimes listed as “absorption-type” laws:

$$A + \bar{A} \cdot B = A + B, \quad A \cdot (\bar{A} + B) = A \cdot B$$

4.6 Simplifying Boolean Expressions Algebraically

A Boolean function can always be expressed in canonical SoM or PoM form, but such expressions are rarely the simplest possible. Simpler expressions require fewer gates, which is cheaper and more efficient to manufacture, so simplification (using the laws above, applied step by step) is an important skill.

Worked example: Simplify $F = A + B \cdot C + \bar{C} \cdot (A + B \cdot D)$.

$$\begin{aligned}
 F &= A \cdot A + A \cdot B \cdot C + A \cdot \bar{C} + A \cdot B \cdot D + B \cdot C \cdot \bar{C} + B \cdot D \cdot \bar{C} \\
 &= A(1 + B \cdot C + \bar{C} + B \cdot D) + B \cdot C \cdot \bar{C} + B \cdot D \cdot \bar{C} \\
 &= A + B \cdot D \cdot \bar{C}
 \end{aligned}$$

(using $A \cdot A = A$, $A \cdot 1 = A$, and $B \cdot C \cdot \bar{C} = 0$ from the Complement Law, then absorbing remaining terms into A via the Redundancy Law).

De Morgan’s theorem is frequently combined with the other laws when simplifying expressions that contain complemented sums or products, e.g. $\bar{A} + \bar{B}$ must first be rewritten as $\overline{A \cdot B}$ before further simplification can proceed algebraically.

4.7 Karnaugh Maps (K-Maps)

A **Karnaugh map** is a graphical, gate-level minimization technique: it represents a truth table as a grid of cells, where each cell corresponds to exactly one minterm, arranged so that any two *physically adjacent* cells (horizontally or vertically, including wrap-around between opposite edges of the map) differ in exactly one variable. If the well-defined grouping rules below are followed correctly, a K-map is guaranteed to produce the simplest possible Sum-of-Products (or Product-of-Sums) expression.

For a function of n variables, the K-map has 2^n cells (e.g. a 3-variable K-map has 8 cells, typically arranged as a 2×4 grid).

4.7.1 K-Map Grouping Rules

1. **Single type only:** a group must contain either all 1s (if deriving a Sum-of-Products expression) or all 0s (if deriving a Product-of-Sums expression) – never a mix.
2. **Rectangular, not diagonal:** groups must be formed by expanding horizontally or vertically; diagonal groupings are not allowed.
3. **Size must be a power of two:** a group may only contain 2^n cells (i.e. 1, 2, 4, 8, ...cells), for $n \geq 0$.
4. **Maximum size:** every group should be expanded to be as large as possible (subject to the other rules).
 - Groups *may overlap* with each other if doing so allows a group to reach its maximum possible size.
 - The map **wraps around:** the top row is considered adjacent to the bottom row, and the leftmost column is considered adjacent to the rightmost column, so groups may be formed across these edges.
5. **Fewest groups:** use as few groups as possible while still covering every required cell and keeping every group at its maximum size.

Grouping 2^k cells together eliminates exactly k variables from the resulting term, because those k variables take on every possible combination of values within the group while the remaining (unchanging) variables define the simplified term for that group. This is precisely why groups must have size 2^k and be formed only along rows/columns (never diagonally) – diagonal or non-power-of-two groupings would not correspond to eliminating a whole variable cleanly across all its possible values.

4.7.2 Don't Care Conditions

Sometimes a function is **incompletely specified**: certain input combinations either cannot occur in practice, or their output value simply does not matter. These input patterns are called **don't care conditions**, marked with an 'X' (or 'd') in the K-map. When grouping, an 'X' can be treated as either a 0 or a 1 – whichever choice produces the largest, most efficient groupings – since its actual value does not affect correctness.

4.7.3 Sum of Products vs Product of Sums from a K-Map

- To derive a simplified **Sum of Products** expression: group the **1s**.
- To derive a simplified **Product of Sums** expression: group the **0s**, then apply De Morgan's theorem appropriately to each group (each group of 0s gives one *sum* term in the final product).

4.8 Functional Completeness and Universal Gates

AND, OR and NOT are considered the three “primary” Boolean functionalities, since every possible Boolean function can be built from some combination of them. A set of gates that can, between them, implement all three of AND, OR and NOT is called a **functionally complete set**. By definition, {AND, OR, NOT} is functionally complete.

Other functionally complete sets include:

- {AND, NOT} – since $A + B = \overline{\overline{A} \cdot \overline{B}}$ (OR can be built from AND and NOT, via De Morgan's theorem).

- {OR, NOT} – since $A.B = \overline{\overline{A} + \overline{B}}$ (AND can be built from OR and NOT, via De Morgan's theorem).
- {NAND} alone – NAND is **individually** functionally complete.
- {NOR} alone – NOR is **individually** functionally complete.

Because NAND and NOR are each individually functionally complete, they are called **Universal Gates**: any Boolean function whatsoever can be implemented using *only* NAND gates, or *only* NOR gates. This matters practically because manufacturing an IC containing only one type of gate is cheaper and simpler than manufacturing ICs with a mixture of gate types (which can also leave unused gates wasted on the chip).

Building NOT, AND, OR from NAND alone:

- NOT: tie both inputs of a NAND gate together: $\overline{A.A} = \overline{A}$.
- AND: a NAND gate followed by a NAND-based NOT (inverter): $\overline{\overline{A.B}} = A.B$.
- OR: invert both inputs first (using NAND-based NOT), then feed them into a NAND gate: $\overline{\overline{A}.\overline{B}} = A + B$ (by De Morgan's theorem).

Building NOT, AND, OR from NOR alone:

- NOT: tie both inputs of a NOR gate together: $\overline{A + A} = \overline{A}$.
- AND: invert both inputs first (using NOR-based NOT), then feed them into a NOR gate: $\overline{\overline{A} + \overline{B}} = A.B$ (by De Morgan's theorem).
- OR: a NOR gate followed by a NOR-based NOT (inverter): $\overline{\overline{A + B}} = A + B$.

When converting a circuit built from mixed gate types into an all-NAND (or all-NOR) circuit by direct substitution, any place where two inversions cancel out (double negation) can be removed to simplify the resulting circuit.

5 Logic Circuits

Logic circuits are broadly classified into two categories:

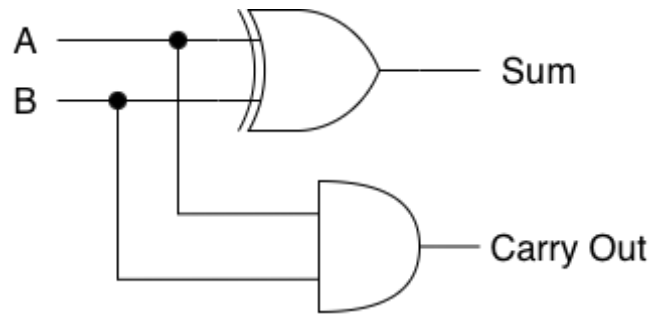
- **Combinational logic circuits**: the output depends only on the current values of the inputs; there is no memory of past inputs.
- **Sequential logic circuits**: the output depends on both the current inputs and the circuit's previous state, which is held in memory elements.

5.1 Combinational Circuit Design

5.1.1 Adders

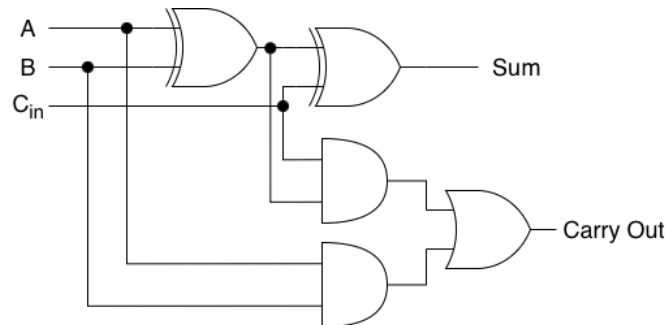
Half Adder:

- Adds two single bits, A and B .
- Cannot accept a carry-in from a previous stage, so it produces two outputs: Sum and Carry Out.
- $\text{Sum} = A \oplus B$, $\text{Carry Out} = A.B$



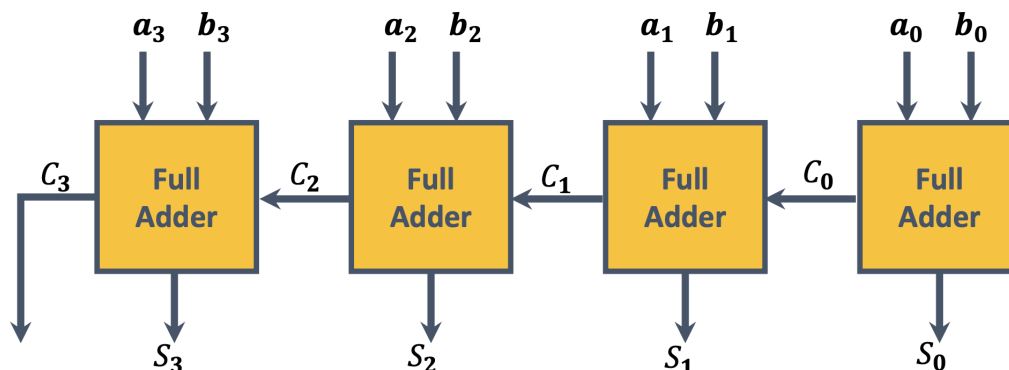
Full Adder:

- Adds three bits: A , B , and a carry-in C_{in} (the carry produced by the previous less-significant stage).
- Can be built by combining two half adders and an OR gate, or directly from AND/OR/XOR gates as shown below.
- $\text{Sum} = A \oplus B \oplus C_{in}$
- $\text{Carry Out} = A.B + A.C_{in} + B.C_{in}$



Ripple Carry Adder:

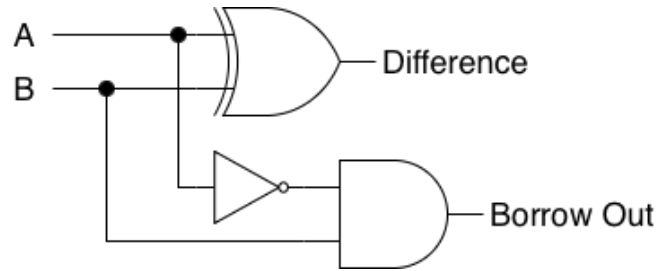
- Chains multiple full adders together to add multi-bit binary numbers.
- For n -bit numbers, n full adders are required, with the carry-out of each stage feeding into the carry-in of the next more-significant stage.
- Called “ripple carry” because the carry signal must propagate (ripple) through every stage in sequence, which limits the speed of the adder for large bit-widths.



5.1.2 Subtractors

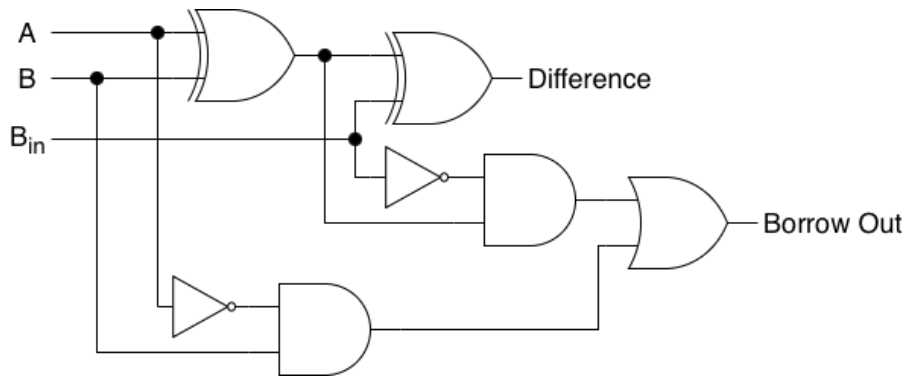
Half Subtractor:

- Subtracts one bit B from one bit A , producing a Difference and a Borrow Out.
- Cannot accept a borrow-in from a previous stage.
- Difference = $A \oplus B$, Borrow Out = $\bar{A}.B$



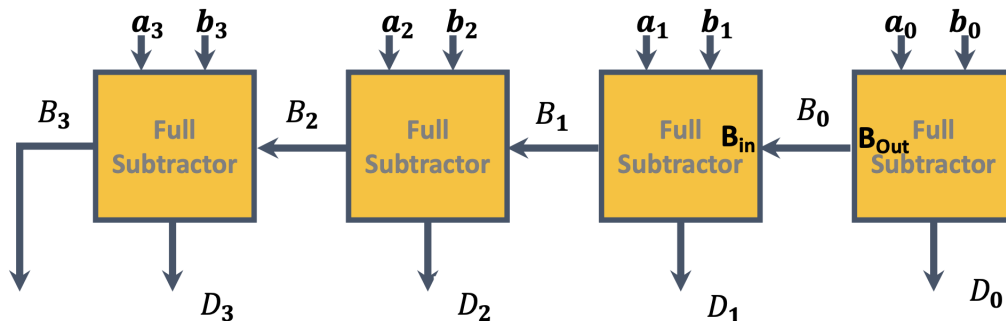
Full Subtractor:

- Subtracts three bits: A , B , and a borrow-in B_{in} .
- Can be built from two half subtractors and an OR gate.
- Difference = $A \oplus B \oplus B_{in}$
- Borrow Out = $\bar{A}.B + \bar{A}.B_{in} + B.B_{in}$



Ripple Borrow Subtractor:

- Chains multiple full subtractors together to subtract multi-bit binary numbers.
- For n -bit numbers, n full subtractors are required, with the borrow-out of each stage feeding into the borrow-in of the next more-significant stage.



5.1.3 Multipliers

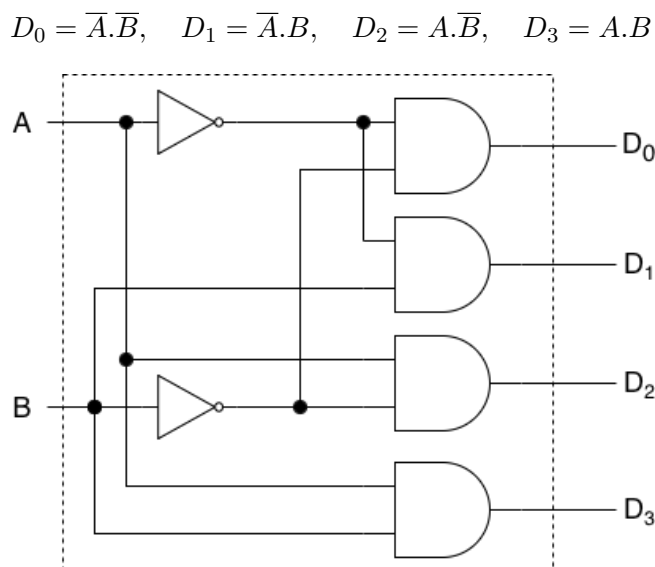
Binary multiplication of two bits mirrors long multiplication:

- **Generation of partial products:** each bit of one operand multiplies (logically ANDs with) every bit of the other operand.
- **Summation of partial products:** the partial products are added together, each one shifted one bit position further to the left than the previous.
- In hardware: AND gates generate each partial product bit, and a network of full adders (arranged similarly to a ripple carry adder) sums the shifted partial products; the physical wiring pattern implements the required left-shifting.

5.1.4 Decoders

- A **decoder** interprets an encoded input (a code word) and activates exactly one of several output lines corresponding to that code.
- For n input lines, a decoder can support up to 2^n unique output lines (e.g. 2 inputs and $2^2 = 4$ outputs is called a 2×4 decoder).
- Most decoders include an **Enable (E)** input; when Enable is 0 the entire circuit is disabled and all outputs remain 0 regardless of the other inputs.
- Decoders are useful **circuit builders**, because each output line of an n -to- 2^n decoder is exactly one minterm of the n inputs. Any Boolean function expressed in Sum-of-Minterms form can therefore be built by feeding the appropriate decoder outputs into an external OR gate.

2×4 **Decoder** (inputs A, B ; outputs D_0 – D_3):

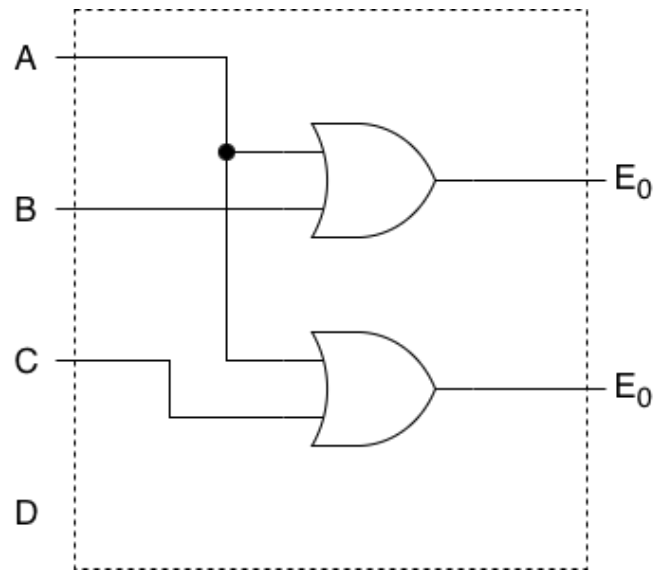


5.1.5 Encoders

- An **encoder** performs the reverse operation of a decoder: it converts an "active" input line back into its corresponding binary code.
- For n output channels, an encoder requires 2^n input channels (e.g. 4 inputs and 2 outputs is called a 4×2 encoder).
- A basic encoder normally assumes exactly one input is active at a time; if more than one input can be active simultaneously, a **priority encoder** is needed to resolve which input takes precedence.

4×2 **Encoder** (inputs A, B, C, D ; outputs E_1, E_0):

$$E_0 = A + B, \quad E_1 = A + C$$

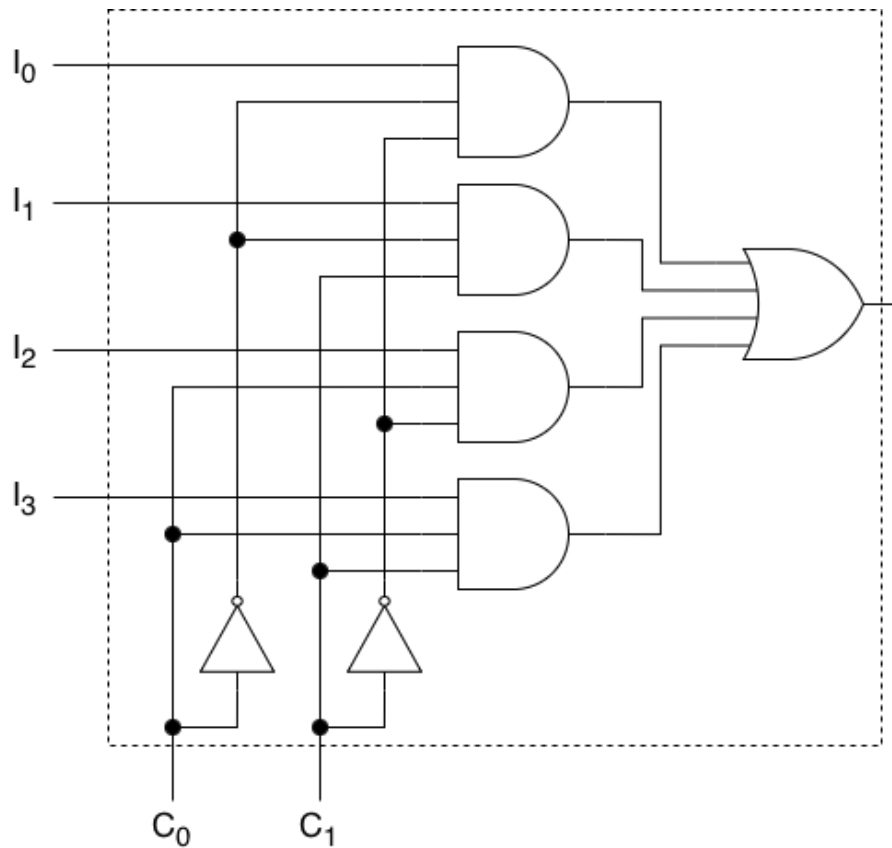


5.1.6 Multiplexers (MUX)

- A **multiplexer** is a digital switch: it has several data input lines but only a *single* output line.
- Its purpose is to select exactly one of the input signals and forward it (route it) to the output.
- Selection is performed using dedicated **control (select) lines**; each unique combination of control-line values acts like a “passcode” that selects one specific input.
- For 2^n data inputs, n control lines are required.

4×1 **Multiplexer** (inputs I_0 – I_3 ; control lines C_0, C_1):

Input	Control Code		Input Activation Condition
	C_0	C_1	
I_0	0	0	$I_0 \cdot \overline{C_0} \cdot \overline{C_1}$
I_1	0	1	$I_1 \cdot \overline{C_0} \cdot C_1$
I_2	1	0	$I_2 \cdot C_0 \cdot \overline{C_1}$
I_3	1	1	$I_3 \cdot C_0 \cdot C_1$



The overall output is the OR of all four activation terms, so only the selected input's value can reach the output at any given time (all other AND gates output 0 because their control-line condition is not satisfied).

5.2 Sequential Logic Circuits

- In a sequential circuit, the output depends on both the present inputs *and* the previous outputs, which are held in a **memory element**.
- The memory element's transition between states is governed by its current state and its inputs; a circuit can therefore be viewed as a time sequence of inputs, outputs and internal states.

5.2.1 Synchronous vs Asynchronous Sequential Circuits

Synchronous Sequential Circuits:

- State changes occur only at discrete time intervals, governed by a global **clock** signal.
- All memory elements update simultaneously, on either the rising or falling edge of the clock pulse.
- *Advantages:* easy to design since all components change state together; hazards and race conditions are minimised since every operation must wait for a clock edge; easy to build complex circuits (e.g. CPUs) because the timing constraint is global and uniform.
- *Disadvantages:* overall speed is limited by the *slowest* path in the circuit; higher power consumption from distributing the clock signal everywhere; in large systems it becomes difficult to ensure the clock reaches every component at exactly the same instant (clock skew).

Asynchronous Sequential Circuits:

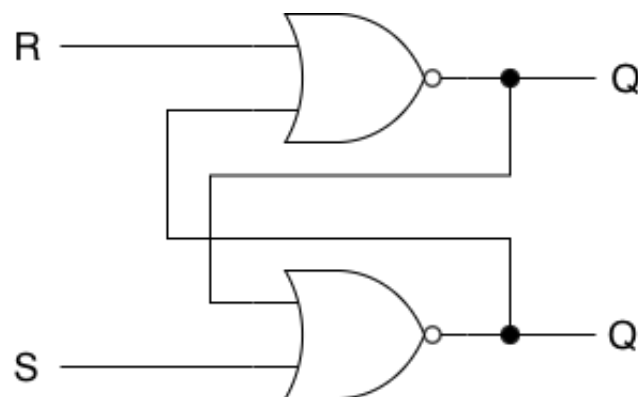
- State changes occur immediately in response to input changes; there is no global clock.
- *Advantages*: faster, since (without a global clock constraint) the circuit can operate at the maximum speed allowed by its individual components; lower power consumption, since only the components actually involved in an operation consume energy; the problem of synchronising a clock signal across the whole circuit is eliminated entirely.
- *Disadvantages*: harder to design correctly; circuits may become unstable since different signals can arrive at different times (leading to glitches/hazards).

5.2.2 Latches

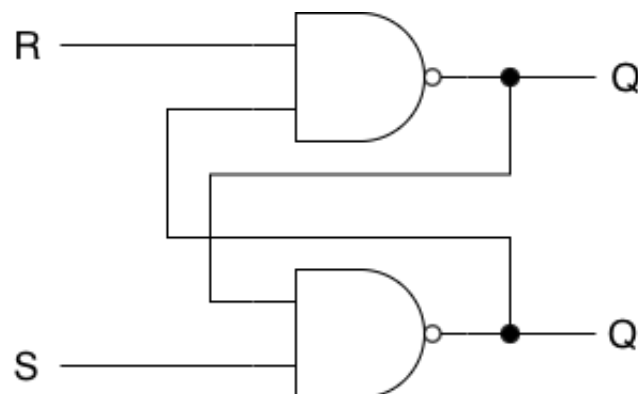
A **latch** is **level sensitive** and **transparent**: as long as its enable signal is active (high), the output continuously tracks the input, and changes immediately whenever the input changes. Latches are asynchronous devices (they do not operate on clock edges), and each is able to store one bit.

S-R Latch (built from NOR gates): the most basic memory building block, with two inputs, Set (S) and Reset (R), and two (complementary) outputs Q and Q' .

S	R	Q	
0	0	Q	No Change
0	1	1	Set
1	0	0	Reset
1	1	0*	Forbidden

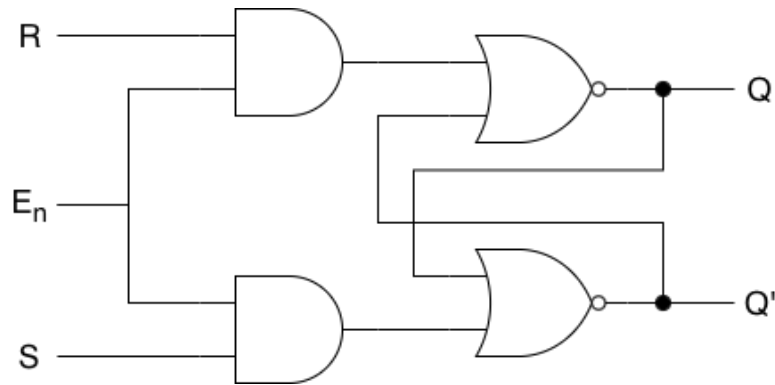


S-R Latch (built from NAND gates), an equivalent circuit using active-low inputs, with its own (dual/inverted) truth-table behaviour:



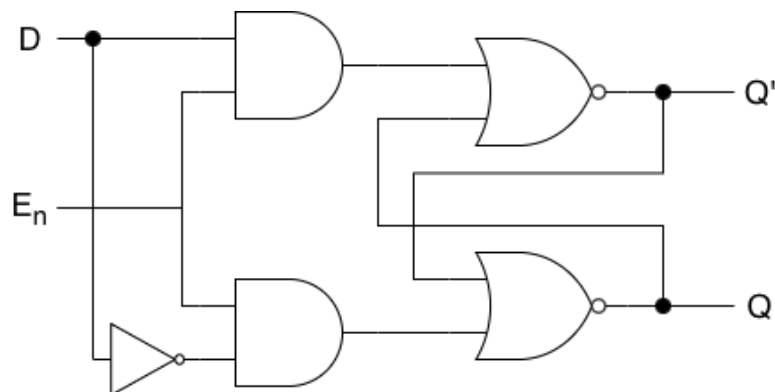
The $S = R = 1$ (NOR version) / $S = R = 0$ (NAND version) case is **forbidden**: it forces Q and Q' to both be equal (violating the requirement that they be complementary), and the resulting next state becomes unpredictable (a race condition) when both inputs are released simultaneously.

S-R Latch with Enable (E_n): adds a control (enable) input so that the latch only responds to S and R while E_n is active (asserted); when E_n is 0 the latch holds its current state regardless of S and R .



D Latch: improves on the S-R latch by eliminating the forbidden-state problem. It has a single Data input (D) and an Enable input (E_n); because D is fed (in complemented and uncomplemented form) into what were the S and R inputs, S and R can never both be asserted at once, so the forbidden state cannot occur.

E_n	D	Q
0	X	Q No Change
1	0	0
1	1	1

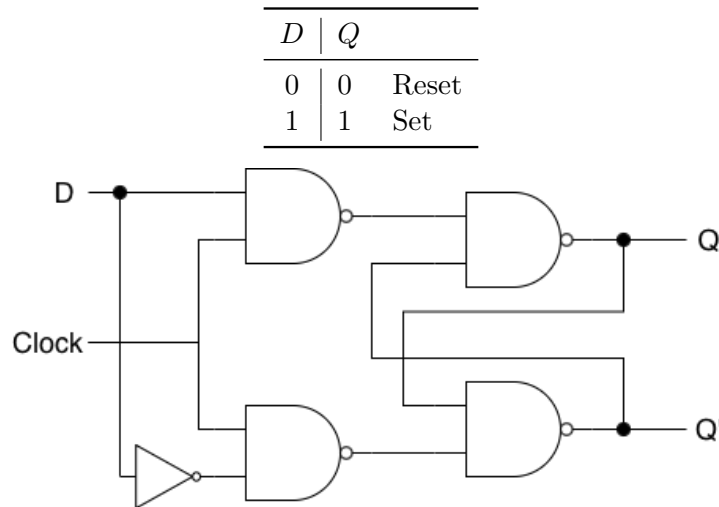


The value of D only reaches the output while the circuit is enabled; when disabled, the output remains unchanged (holds its last value).

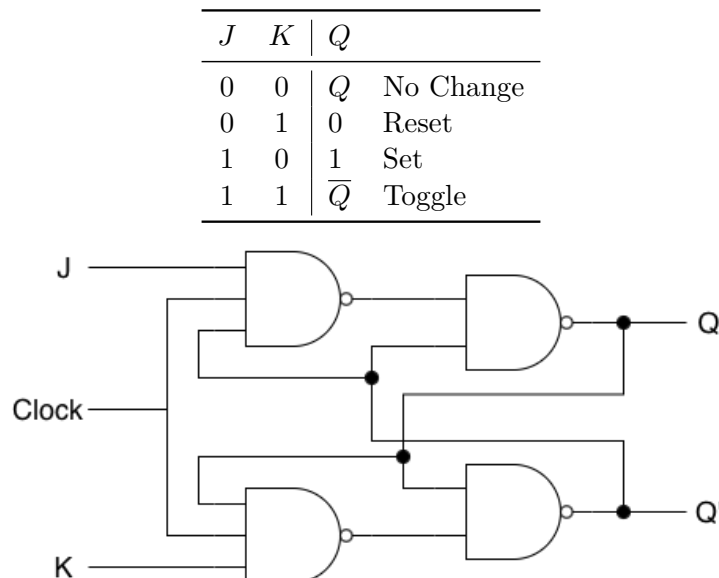
5.2.3 Flip-Flops

A **flip-flop** is designed to be more stable than a latch, because it is **edge-triggered** rather than level-sensitive: instead of responding continuously whenever the clock/enable signal is high, it responds only at the instant of a clock **transition** (edge) – either the **positive edge** (rising, $0 \rightarrow 1$) or the **negative edge** (falling, $1 \rightarrow 0$). This edge sensitivity solves the problem where a level-sensitive latch could produce multiple output changes during a single active clock pulse if its input changed while enabled. Like a latch, a flip-flop stores one bit.

D Flip-Flop: the most widely used flip-flop in digital systems. It acts like a controlled buffer: it captures whatever value is present on the D (Data) input and transfers it to output Q at the next active clock edge.

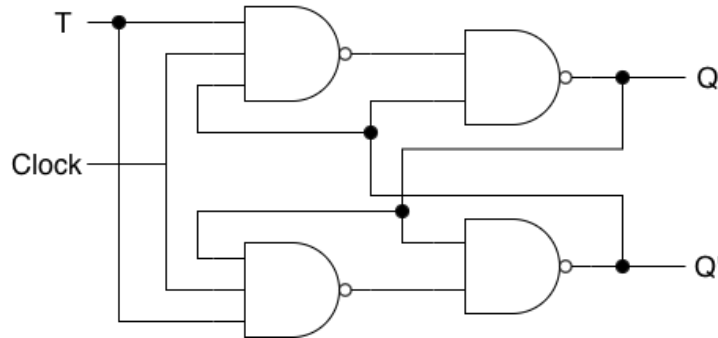


J-K Flip-Flop: an enhanced version of the S-R latch concept, adding a feedback mechanism from the outputs back to the input gates so that the previously-forbidden input combination is instead repurposed to make the flip-flop **toggle** its state.



T Flip-Flop: a simplified special case of the J-K flip-flop in which the J and K inputs are tied together and driven by a single input, T (Toggle).

T	Q	
0	Q	No Change
1	$\overline{Q}$	Toggle



Because it toggles on every active clock edge when $T = 1$, the T flip-flop is especially useful for building binary counters.

5.2.4 Registers and Counters

- A **register** is a group of flip-flops (typically D flip-flops), one per bit, sharing a common clock, used to store a multi-bit value (e.g. an 8-bit register holds one byte, built from 8 D flip-flops).
- A **counter** is a register (built from flip-flops, e.g. T or J-K flip-flops) whose state cycles through a fixed sequence of values (typically a binary count sequence) on each active clock edge/toggle condition.
 - **Ripple (asynchronous) counters:** each flip-flop's clock input is driven by the *output* of the previous flip-flop rather than by a common clock; this is simple to build but the count "ripples" through the stages with cumulative delay, similar to a ripple carry adder.
 - **Synchronous counters:** every flip-flop shares the same common clock signal, with additional combinational logic determining each flip-flop's next state; this avoids the cumulative ripple delay, at the cost of extra logic gates.

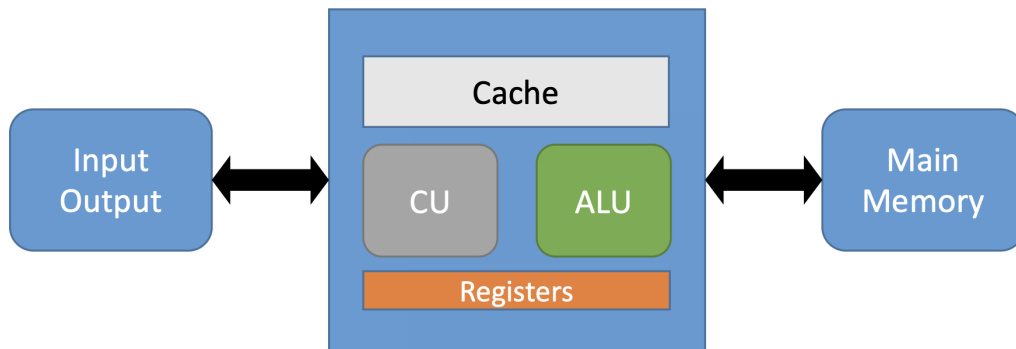
6 The CPU and Its Organization

6.1 Key Components

- **ALU (Arithmetic Logic Unit):** performs mathematical and logical operations.
- **CU (Control Unit):** fetches instructions from main memory, decodes them, determines which operations the ALU should perform, and controls the flow of data through the system.
- **Registers:** small, very fast storage locations that can hold binary information – much faster to access than main memory. They store a wide variety of data such as instructions, data values and addresses. Key registers include:

Register	Name	Purpose
MAR	Memory Address Register	Holds the address of the memory location to be accessed.
MDR	Memory Data Register	Holds data being transferred to or from memory.
AC	Accumulator	Stores intermediate arithmetic and logic results.
PC	Program Counter	Contains the address of the next instruction to execute.
CIR	Current Instruction Register	Contains the current instruction while it is being processed/decoded.

- **Cache:** a small, very fast memory that stores the most frequently (or recently) used data, to reduce the average time needed to access memory.
- **Input/Output interfaces:** allow the CPU to communicate with peripheral devices.



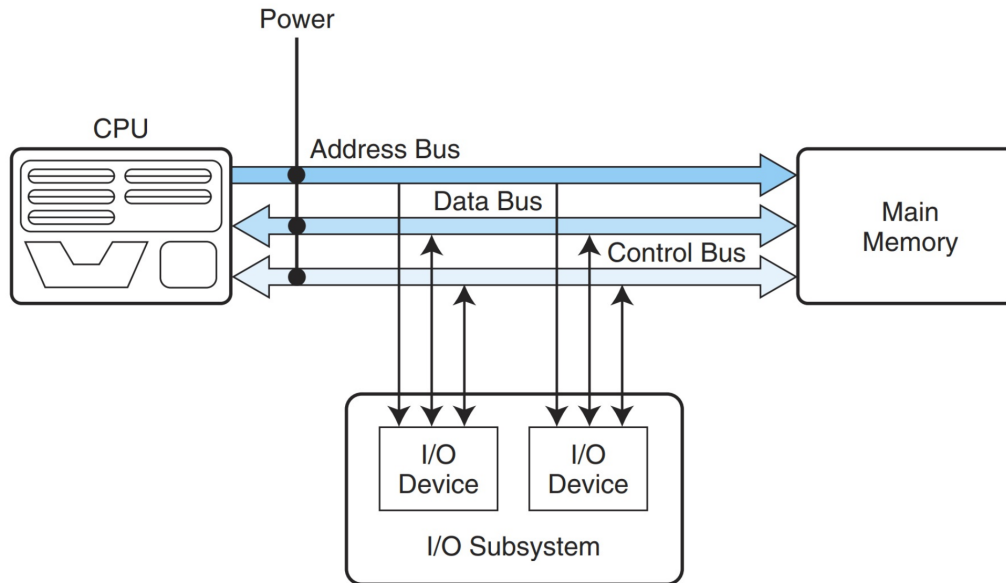
6.2 Main Memory and Addressing

- Main memory is organised as a sequence of individually addressable storage locations.
- **Byte-addressable** memory: each individual byte has its own unique address (the most common scheme in modern systems).
- **Word-addressable** memory: each address refers to one whole word (which may span several bytes), rather than each individual byte.
- The number of bits in an address determines the maximum number of distinct memory locations that can be addressed: with n address bits, up to 2^n locations can be addressed.

Worked example: A machine has a 256-byte main memory. Since $256 = 2^8$, an address needs at least 8 bits to uniquely identify every byte of memory.

6.3 Buses

- A **bus** is a shared set of wires that connects multiple subsystems, acting as a path along which data can travel between them.
- Only one device can use a given bus at any instant, since sharing it among multiple simultaneous transfers would create bottlenecks/collisions.
- Bus speed (effective bandwidth) depends on both the physical **width** of the bus (number of parallel wires/bits it can carry at once) and the number of devices that must share it.



- **Address Bus:** carries the address of the memory location (or device) that the CPU wants to read from or write to. It is **unidirectional** (one-directional): addresses only ever travel from the CPU outward.
- **Data Bus:** carries the actual data being transferred. It is **bidirectional**: if the CPU issues a read request, data flows from main memory to the CPU; if the CPU issues a write request, data flows from the CPU to main memory.
- **Control Bus:** carries control signals, such as read/write command lines, interrupt requests, and other coordination signals. It is bidirectional.

6.4 Clock and Clock Speed

- The CPU's **clock** generates a regular sequence of pulses that synchronise the operations of a synchronous digital system.
- **Clock speed** (clock rate) is measured in Hertz (Hz), i.e. cycles per second (commonly given in GHz, billions of cycles per second, for modern processors).
- **Clock cycle time** is the reciprocal of clock speed: $T = 1/f$. A higher clock speed means a shorter clock cycle time, and (generally) faster execution – though actual performance also depends heavily on how many clock cycles each instruction requires.

6.5 CPU Performance

- **CPU time** (execution time) for a program can be expressed as:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

or equivalently

$$\text{CPU Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

where **CPI** (Cycles Per Instruction) is the average number of clock cycles required to execute one instruction.

- **Instruction count** can be considered in two ways:
 - **Static instruction count:** the number of distinct instructions that appear in the compiled program (the size of the program's machine code).

- **Dynamic instruction count:** the number of instructions actually *executed* while the program runs (which can differ substantially from the static count due to loops and branches being executed multiple times, or code paths being skipped).
- If a program's instruction mix consists of several instruction types, each with its own CPI, then the overall (average) CPI is the weighted average:

$$\text{CPI} = \sum_i \left(\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Total Instruction Count}} \right)$$

- *Worked example:* Suppose a program has three instruction types with the following counts and CPIs – Type A: 20,000 instructions at CPI 3; Type B: 30,000 instructions at CPI 4; Type C: 50,000 instructions at CPI 5 – and the clock rate is 2 GHz.

Total instruction count = 20,000 + 30,000 + 50,000 = 100,000

Total cycles = (20,000 × 3) + (30,000 × 4) + (50,000 × 5)
= 60,000 + 120,000 + 250,000 = 430,000 cycles

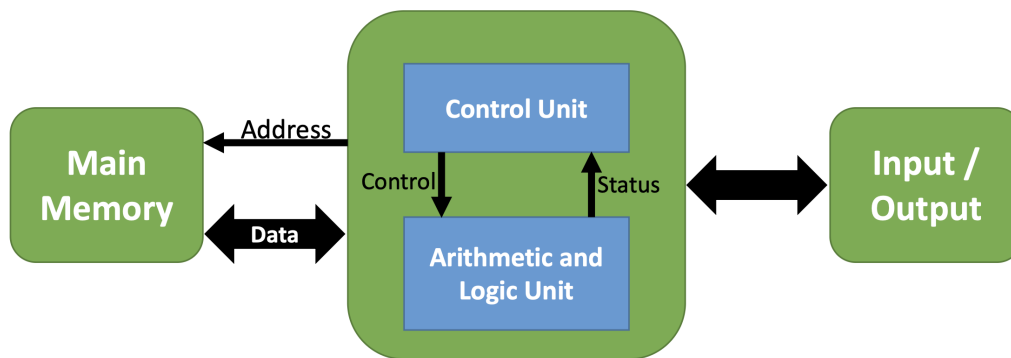
Average CPI = $\frac{430,000}{100,000} = 4.3$

CPU Time = $\frac{430,000}{2 \times 10^9} = 215 \mu\text{s}$

- Reducing CPU time can be achieved by: reducing the instruction count (better compiler optimisation or a more efficient algorithm/ISA), reducing the average CPI (a more efficient microarchitecture, e.g. pipelining), or reducing the clock cycle time (increasing the clock rate, subject to physical/power limits).

6.6 Von Neumann Architecture

The Von Neumann Architecture (see Section 1.4) organises a computer around three principal blocks – Main Memory, the CPU (containing the Control Unit and the Arithmetic and Logic Unit), and Input/Output – connected as shown below.



- The **Address** line carries the memory location the CPU wants to access (sent from the CPU to Main Memory).
- The **Data** line carries data between Main Memory and the CPU, and is bidirectional.
- Within the CPU, the **Control Unit** sends **Control** signals to the ALU telling it what operation to perform, and receives **Status** signals back from the ALU (e.g. flags indicating overflow, zero result, etc.).
- Input/Output devices connect to the rest of the system and exchange data with it.

6.7 The Fetch-Decode-Execute Cycle

Every instruction executed by the CPU passes through this repeating cycle:

1. **Fetch:** the CPU fetches the next instruction to execute from memory (using the address held in the Program Counter), and loads it into the appropriate register (the Current Instruction Register).
2. **Decode:** the CPU decodes the fetched instruction; any required operands are fetched from memory and placed into registers.
3. **Execute:** the CPU executes the instruction (e.g. the ALU performs the required operation).
4. The CPU **updates the Program Counter** (normally incrementing it to point at the next instruction, unless the executed instruction was itself a jump/branch).
5. The cycle **repeats** from the top for the next instruction.

6.8 Instruction Set Architecture (ISA)

- The **Instruction Set Architecture** is the set of machine-specific instructions that a given computer/processor can execute, together with the precise format (encoding) of each instruction.
- *Worked example:* Consider a hypothetical machine architecture with a 256-byte main memory, 16 general-purpose registers, 16-bit instructions, and 8-bit integers/floating-point values.
 - Since the instruction is 16 bits wide and there must be some number of bits allocated to the **opcode** (the operation code identifying which instruction it is) and some to operands/addresses:
 - With 16 registers, uniquely identifying one register requires $\log_2(16) = 4$ bits per register address.
 - If, say, an instruction format allocates 4 bits to the opcode (allowing up to $2^4 = 16$ distinct instructions) and the remaining 12 bits are split between one or more register/operand fields (e.g. two 4-bit register fields and a further 4-bit field, or one 4-bit register field plus an 8-bit immediate/address field, depending on the instruction type), the exact split depends on the instruction's format/type.
- **Instruction types** generally fall into three broad categories:
 - **Data transfer instructions:** move data between registers, or between registers and memory (e.g. LOAD, STORE, MOVE).
 - **Data processing (arithmetic/logic) instructions:** perform computation (e.g. ADD, SUB, AND, OR, shift operations).
 - **Program control instructions:** alter the normal sequential flow of execution (e.g. unconditional and conditional jumps/branches, subroutine call/return instructions).

7 Addressing Modes and Instruction-Level Pipelining

7.1 Addressing Modes

Addressing modes specify *where* the operand(s) of an instruction are located – an addressing mode can specify a constant, a register, or a location in memory. A typical instruction format reserves a fixed number of bits for the opcode, and the remaining bits are interpreted differently depending on the addressing mode – e.g. an opcode field followed by a register field and

then either an immediate value, a memory address, or additional register fields. Real computer architectures typically provide *multiple* of the addressing modes below, so that the most efficient one can be chosen for each situation.

7.1.1 Effective Address and Physical Address

- The **effective address** of an operand is the location where the actual operand value can be found.
 - In a system *without* virtual memory, the effective address is either a main memory address or a register.
 - In a system *with* virtual memory, the effective address is a virtual address or a register.
- The **physical address** is the real hardware address in main memory. Translating from an effective/virtual address to the physical address is the responsibility of the **Memory Management Unit (MMU)**, a hardware device that manages and controls access to memory. Because the MMU handles this translation automatically, the process is invisible to the programmer.

7.1.2 Immediate Addressing

- The instruction itself contains the operand's actual value; the opcode is immediately followed by the value to be used, so the data item is available as part of the instruction.
- *Example:* `LI R1, 0x0A` – load register R1 with the immediate value 0x0A.
- **Pros:** very fast, since the value is already included in the instruction (no extra memory access needed to fetch it).
- **Cons:** less flexible, since the value is fixed at compile time; the size of the representable number is restricted by the size of the instruction's address/operand field.

7.1.3 Direct Addressing

- The instruction's address field directly contains the effective address of the operand.
- *Example:* `LOAD R1, 0x80` – load register R1 with the contents of memory location 0x80.
- **Pros:** fast, since the value can be accessed with a single memory reference; more flexible than immediate addressing since the value itself is not baked into the instruction.
- **Cons:** provides only a limited address space, since the address field width restricts how much memory can be directly addressed.

7.1.4 Indirect Addressing

- The instruction's address field contains a **pointer** to the effective address of the operand (i.e. the address field holds the address of the address).
- *Example:* given an array starting at address 0x1000, with register R1 holding the base address (0x1000) and R2 holding an index (e.g. 2), the effective address of the desired element is first computed: `ADD R3, R1, R2`. The value at that computed address is then loaded using indirect addressing: `LD R4, (R3)` – the parentheses around R3 indicate a reference to the value pointed to by the address stored in R3 (directly analogous to pointer dereferencing in C, e.g. `int value = *ptr;`).

- **Pros:** allows access to a larger memory space, since an entire word/register can hold a full address.
- **Cons:** executing the instruction requires **two** memory references – one to fetch the address, and a second to fetch the actual operand value – making it slower than direct addressing.

7.1.5 Register Addressing (Register Direct)

- Similar in concept to direct addressing, but a **register** (rather than a memory location) holds the operand.
- *Example:* ADD R1, R2, R3 – all operands are registers.
- **Pros:** very fast, since accessing a register is much quicker than accessing memory; no memory reference is required at all.
- **Cons:** limited by the small number of available registers, restricting how much data can be held this way at once.

7.1.6 Register Indirect Addressing

- A variant of indirect addressing: the instruction's operand specifies a **register** that holds a memory address (the effective address) of the actual data item to be used (rather than the address field of the instruction pointing to a memory location holding a pointer).
- **Pros:** only a single memory reference is required to fetch the operand (faster than full indirect addressing, since the pointer itself is already in a register rather than needing to be fetched from memory).
- **Cons:** still limited address space compared to full indirect addressing, since it relies on a fixed number of registers to hold pointers.

7.1.7 Displacement Addressing

- A combination of direct addressing and register indirect addressing.
- The instruction contains **two** address fields: one explicit (a direct address or constant offset) and one implicit/register-based; the effective address is calculated by adding the register's contents to the given displacement/offset value.
- This is the basis of common addressing techniques such as *indexed addressing* and *base-register addressing* used to access arrays and structures efficiently.

7.1.8 Stack Addressing

- The operand is implicitly located on top of a hardware **stack**, accessed via a dedicated Stack Pointer register, rather than through an explicit address field in the instruction.
- Common in architectures that support subroutine calls (storing return addresses) and expression evaluation.

7.1.9 Worked Comparison Example

Suppose memory location 0x100 contains the value 0x210; location 0x210 contains the value 0x350; and location 0x350 contains the value 0x050. For the instruction LOAD R1, 0x100:

Addressing Mode of the 0x100 operand field	Value loaded into R1
Immediate	0x100 (the literal value given in the instruction)
Direct	0x210 (the contents of memory location 0x100)
Indirect	0x350 (the contents of the location <i>pointed to by</i> 0x100)

7.2 Instruction-Level Pipelining

- Modern CPUs divide the fetch–decode–execute cycle into a series of small, independent steps.
- Because these steps are made independent of one another, different steps *belonging to different instructions* can be carried out in parallel – this technique is called **pipelining**.
- The core idea: divide the execution of a single instruction into several sequential stages, then execute the corresponding stage of several *different* (consecutive) instructions simultaneously, much like an assembly line.

An example set of pipeline stages:

1. Fetch instruction
2. Decode opcode
3. Calculate the effective address of the operand(s)
4. Fetch operand(s)
5. Execute instruction
6. Store result

Instruction-level pipelining overlaps these stages across a set of consecutive instructions (e.g. while instruction 2 is being decoded, instruction 1 can already be having its effective address calculated, and instruction 3 can simultaneously begin being fetched). For maximum throughput, the time taken by each pipeline stage should ideally be balanced (roughly equal), since the overall pipeline can only advance as fast as its slowest stage.

7.2.1 Pipelining Efficiency and Speedup

Let there be a **k -stage** pipeline, with clock cycle time t_p (the time taken per stage), and n instructions to be processed.

- The first instruction, I_1 , must pass through all k stages before it completes, requiring $k \times t_p$ time.
- Because the pipeline is now full, each of the remaining $n - 1$ instructions emerges (completes) from the pipeline at a rate of one per clock cycle, taking a further $(n - 1)t_p$ time in total.
- **Total time to complete n instructions using a k -stage pipeline:**

$$k \times t_p + (n - 1)t_p = (k + n - 1)t_p$$

Speedup compares the pipelined execution time to the time a non-pipelined CPU would take (which would need $k \times t_p$ time *for every* instruction, i.e. $k \times t_p \times n$ total):

$$\text{Speedup} = \frac{k \times t_p \times n}{(k + n - 1)t_p} = \frac{k \cdot n}{k + n - 1}$$

As n becomes very large, this speedup approaches k (the number of pipeline stages) – i.e. the theoretical maximum benefit of pipelining is proportional to the number of stages, though this ideal is rarely fully achieved in practice.

7.2.2 Advantages, Disadvantages, Limitations and Issues

- **Advantages:** significantly increases instruction throughput (instructions completed per unit time) without requiring a faster clock; makes efficient use of CPU hardware resources, since different functional units can be kept busy processing different instructions' stages simultaneously.
- **Disadvantages:** significantly increases hardware design complexity; latency for any single instruction is not reduced (an individual instruction still takes k stages to complete) – pipelining improves throughput, not the completion time of one instruction.
- **Limitations:** the pipeline can only run as fast as its *slowest* stage, so stage durations must be carefully balanced; deeper pipelines (larger k) give diminishing practical returns due to overheads.
- **Issues (hazards):** pipelining introduces **hazards** that can stall or disrupt the pipeline, such as:
 - **Structural hazards:** two instructions in different stages need the same hardware resource at the same time.
 - **Data hazards:** an instruction depends on the result of a previous instruction that has not yet completed/written back its result.
 - **Control hazards:** caused by branch/jump instructions, where the next instruction to fetch is not known until the branch itself has been resolved, potentially wasting the work already done on incorrectly fetched instructions.

8 Memory Organization

8.1 Primary and Secondary Memory

- **Primary Memory:** memory directly accessible to the CPU.
 - Cache Memory
 - Main Memory
- **Secondary Memory:** memory used for longer-term, larger-capacity storage, not directly accessed by the CPU.
 - Magnetic Disks (Hard Disk Drives)
 - Magnetic Tapes
 - USB Sticks / Flash Drives

8.2 Types of Memory

- **Read Only Memory (ROM): non-volatile** (retains its content without power). Variants include:
 - **PROM** (Programmable Read Only Memory)
 - **EEPROM** (Electrically Erasable Programmable Read Only Memory)

- **Random Access Memory (RAM): volatile** (loses its content when power is removed). Variants include:
 - **DRAM** (Dynamic RAM)
 - **SRAM** (Static RAM)

8.3 Flash Memory

- Built from floating-gate metal-oxide-semiconductor field-effect transistors.
- **Non-volatile**, and supports both reading and writing.
- Common uses: memory cards, USB flash drives, solid state drives.

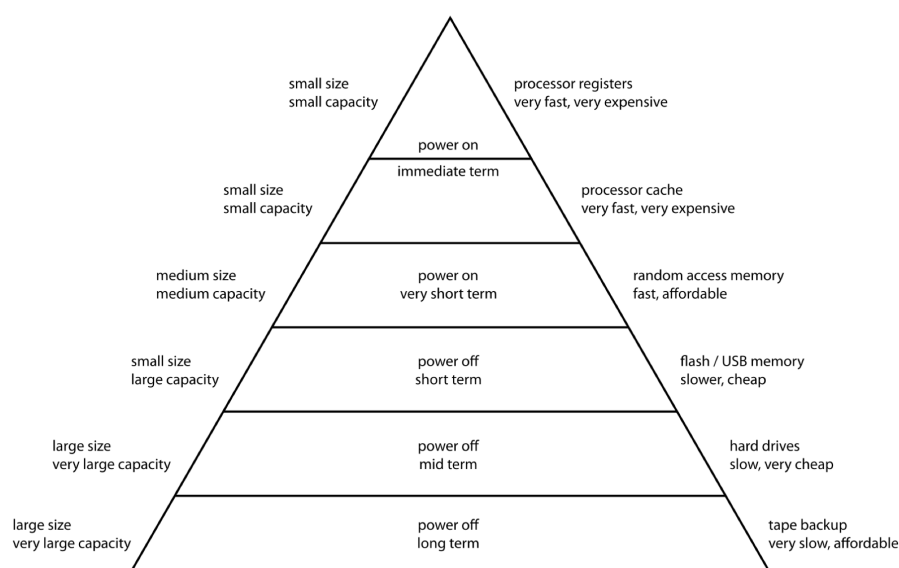
8.4 Solid State Devices (SSD)

- Typically built on flash memory technology.
- Compared to traditional Hard Disk Drives (HDD): more resistant to physical shock, run silently (no moving parts), and offer faster access times.
- **Not** well suited for very long-term storage without power, since charge stored in the memory cells depletes gradually over time.

(3D XPoint is mentioned as a newer memory technology worth further independent reading, positioned between DRAM and flash in terms of speed/persistence characteristics.)

8.5 Memory Hierarchy

Computer storage components are organised into a **hierarchy** based on several attributes: **access time / latency**, **cost** (per unit of storage), **size** (capacity), and **distance from the CPU**. As one moves up the hierarchy (towards the CPU), memory becomes faster and more expensive per byte, but smaller in capacity; moving down, memory becomes slower and cheaper per byte, but much larger in capacity.



From top to bottom of the pyramid: processor registers (smallest, fastest, most expensive, immediate/volatile on power-off) → processor cache → main memory (RAM) → flash/USB memory → hard drives → tape backup (largest, slowest, cheapest, persists long-term without power).

8.6 Typical Data Access Pattern

1. The CPU first requests data from its nearest storage: the **cache**.
2. If the data is not found in the cache, the request proceeds to **main memory**.
3. If still not found, the request proceeds to secondary storage (e.g. **HDD**).
4. Once the data is located, it is loaded (copied) into main memory, and then into cache, so that it (and often, by locality, nearby data) is readily available for subsequent accesses.

8.7 The CPU–Memory Gap

- CPU cycle time has historically decreased (i.e. CPU speed increased) much more rapidly than memory access times have improved.
- Typical relative speed ordering: $\text{HDD} > \text{SSD} > \text{DRAM} > \text{SRAM} \gg \text{CPU}$ (in terms of access latency – HDD is by far the slowest, the CPU is by far the fastest).
- This growing mismatch between CPU speed and memory speed is called the **CPU–Memory Gap**, and it can significantly limit overall system performance if the CPU is frequently left waiting for data.
- The **Principle of Locality** is exploited to help bridge this gap.

8.8 Principle of Locality

The principle of locality describes the tendency of a CPU to access the **same set of memory locations repeatedly over a short period of time**: programs tend to reference data and instructions with addresses that are near, or equal, to addresses they have referenced recently.

- **Spatial Locality**: items with nearby addresses tend to be referenced close together in time (e.g. iterating sequentially through an array).
- **Temporal Locality**: a recently referenced item is likely to be referenced again in the near future (e.g. a loop variable or a frequently-called function).

Caching exploits both forms of locality: bringing in a whole block of nearby data exploits spatial locality, while keeping recently used data in the (small, fast) cache exploits temporal locality.

8.9 Cache Memory

- A cache is a smaller, faster memory space that holds (a copy of) data from a larger, slower memory space.
- Typically resides physically on the CPU chip itself.
- Typical characteristics: **transient** (highly volatile – its content is not meant to persist); may hold frequently referenced data, or data predicted to be referenced in the near future.

8.9.1 Cache Hierarchy

- Cache memories are typically built from **SRAM**, which has a high per-unit cost.
- Because larger and faster cache space is expensive, a **hierarchy of caches** is used instead of one large cache – different levels of cache (e.g. **L1**, L2, L3), where L1 is the smallest and fastest (closest to the CPU core), and each successive level is larger but slower.

8.10 Cache Performance

- **Cache Hit:** the data requested by the CPU is found in the cache.
- **Cache Miss:** the requested data is *not* found in the cache.
- **Cache Hit Ratio (HR):** the probability that a requested item is found in the cache.
- **Cache Miss Ratio:** $1 - \text{Hit Ratio}$.
- **Effective Access Time (EAT):** the average time it takes for the CPU to access data, taking into account both hits and misses:

$$EAT = HR \times T_h + (1 - HR) \times T_m$$

where HR is the Hit Ratio, T_h is the cache access time on a hit, and T_m is the (typically much larger) data access time on a miss (which usually also includes the time to then access main memory).

8.11 Key Design Decisions of a Cache

1. **Cache Volume** (capacity / size).
2. **Write Policy:** Write Back or Write Through.
3. **Mapping Scheme:** Direct Mapped, Fully Associative, or Set Associative (including choice of line/block replacement policy where relevant).

8.11.1 Write Policy

Because a cache is a form of **redundant storage** – the same data conceptually exists in both the cache and in main memory (and may also be shared across multiple processes) – maintaining **consistency** between the cache and main memory becomes a challenge whenever the data is altered (written to). A cache therefore needs a clearly defined write policy:

- **Write Through:** whenever a write operation happens, it is performed on *both* the cache and main memory simultaneously (synchronously).
 - Comparatively simple to implement.
 - Increases bus traffic, since every single write generates a main-memory transaction.
- **Write Back:** a write operation is performed *only* on the cache; the updated data is written back to main memory only later, when that cache entry is eventually **evicted** from the cache.
 - Sometimes called “lazy writes”.
 - More complex to implement (the cache must track which entries have been modified, typically via a “dirty bit”), but reduces bus traffic since repeated writes to the same location only cost one main-memory write (on eviction), not one per write.

8.11.2 Mapping Schemes – Why They Are Needed

The CPU refers to data/instructions by their addresses. Data is searched for in the cache first, so cached data must be retrievable via addresses. However, since the cache is much smaller than main memory, cache locations cannot simply be given the same addresses as main memory locations – a **mapping scheme** is required to translate between a main-memory address and a location within the (much smaller) cache. The three main mapping schemes are:

- Direct Mapped

- Fully Associative
- Set Associative

8.11.3 Cache Lines (Cache Blocks) and Cache Entries

- Data is transferred between main memory and cache in fixed-size blocks called **cache lines** (or cache blocks) – not one byte/word at a time – which exploits spatial locality (nearby data is brought in together).
- When a cache line is copied into the cache, a **cache entry** is created, consisting of the copied data *plus* a **tag** that identifies which memory location(s) this entry corresponds to.
- The amount of data held in a cache entry is always exactly equal in size to a cache line.

8.12 Direct Mapped Cache

- Main memory is conceptually divided into fixed-size blocks.
- Each memory block has a single, pre-defined cache-entry position that it can be placed into; i.e. a given memory block will *always* map to the same cache entry position.
- Since the number of blocks in main memory is far larger than the number of available cache entries, several different memory blocks necessarily map to the *same* cache entry position, and only one of them can occupy that position at any given time.
- A **tag** is stored alongside each cache entry to identify exactly which memory block currently occupies that position.

8.12.1 Address Breakdown: Tag, Index, Offset

A memory address used to access a direct-mapped cache is conceptually split into three fields:

- **Offset**: the low-order bits identifying which specific byte/word *within* a cache line is being referred to.
- **Index**: the next bits, identifying *which* cache entry position (line slot) the address maps to.
- **Tag**: the remaining (high-order) bits, stored with the cache entry, used to verify *which* of the several possible memory blocks currently occupies that cache entry.

Worked example (small illustrative cache): A byte-addressable main memory of 1024 bytes, using a direct-mapped cache with 2 bytes per cache line and a total cache volume of 8 bytes (so there are $8/2 = 4$ cache entry positions).

- Since each cache line holds 2 bytes, **1 bit** is needed for the Offset (to select between the 2 bytes within a line).
- Since there are 4 cache entry positions, **2 bits** are needed for the Index (to select 1 of 4 positions).
- Since main memory is 1024 bytes, a full address needs $\log_2(1024) = 10$ bits; subtracting the 1 offset bit and 2 index bits leaves $10 - 1 - 2 = 7$ bits for the Tag.
- So an address is split as: **Tag (7 bits) | Index (2 bits) | Offset (1 bit)**.

Searching a direct-mapped cache:

1. Use the **Index** bits of the requested address to locate the corresponding cache entry position directly.

2. Use a comparator to check whether the **Tag** stored at that cache entry matches the Tag bits of the requested address.
3. If the tag matches, check the entry's **Valid Bit**:
 - Valid bit = 1: **Cache Hit**.
 - Valid bit = 0: **Cache Miss** (the entry has never been validly loaded).
4. If the tag does *not* match, it is a **Cache Miss** (a different memory block currently occupies that position, so the requested block must be fetched from main memory and will replace/evict the current occupant of that position).

8.12.2 General Formulas for Tag / Index / Offset Lengths

$$\text{Offset length (bits)} = \log_2 \left(\frac{\text{Cache line size}}{\text{Size of one addressable unit}} \right)$$

$$\text{Index length (bits)} = \log_2 \left(\frac{\text{Total cache volume}}{\text{Cache line size}} \right)$$

$$\text{Tag length (bits)} = \text{Total address length (bits)} - \text{Index length} - \text{Offset length}$$

Worked exam-style example: a 2 MB direct-mapped cache, with a cache line size of 32 KB, for a **word-addressable** 1 GB main memory where each word is 32 bits (4 bytes).

- **Offset:** the number of addressable words (not bytes, since memory is word-addressable) within one cache line:

$$\frac{32\text{KB}}{32 \text{ bits}} = \frac{32 \times 2^{10} \text{ Bytes}}{4 \text{ Bytes}} = \frac{2^{15} \text{ Bytes}}{2^2 \text{ Bytes}} = 2^{13} \Rightarrow \mathbf{13 \text{ bits for Offset.}}$$

- **Index:** the number of cache entries (line slots) available:

$$\frac{2\text{MB}}{32\text{KB}} = \frac{2 \times 2^{20} \text{ Bytes}}{2^5 \times 2^{10} \text{ Bytes}} = \frac{2^{21}}{2^{15}} = 2^6 \Rightarrow \mathbf{6 \text{ bits for Index.}}$$

- **Total address length:** number of distinct word-addresses in a 1 GB, word-addressable (4 bytes/word) memory:

$$\frac{1\text{GB}}{32 \text{ bits}} = \frac{2^{30} \text{ Bytes}}{4 \text{ Bytes}} = 2^{28} \Rightarrow \mathbf{28 \text{ bits total address length.}}$$

- **Tag length** = 28 – 6 – 13 = **9 bits**.

28 bits (total address)		
Tag Length	Index Length	Offset Length
9	6	13

Note: if the memory in such a question is **byte-addressable** instead of word-addressable, the offset/index/tag calculations are performed directly in bytes (no division by the word size is needed when computing the offset), which will generally shift how many bits are required for each field – always check carefully whether a question specifies byte or word addressability.

8.12.3 Advantages and Disadvantages of Direct Mapped Cache

- **Advantages:** simple to implement in hardware; fast lookups, since the index bits give the exact position to check with a single comparison (no searching required).
- **Disadvantages:** prone to a high number of cache misses if the program happens to repeatedly access two or more memory blocks that map to the *same* cache position (an effect sometimes called “thrashing”), even if the rest of the cache is empty/unused; is the least flexible mapping scheme.

8.13 Fully Associative Cache

- The direct-mapped scheme enforces a strict mapping rule (each block maps to exactly one position); **fully associative** caching is the complete opposite.
- There are no restrictions: an incoming cache line can be placed *anywhere* currently available in the cache.
- This is a much more flexible mapping scheme (far fewer unnecessary evictions/misses caused purely by mapping collisions).
- The major drawback is that **searching** the cache is expensive: since a block could be anywhere, every single cache entry’s tag potentially needs to be checked (compared) on every access, requiring many parallel comparators.

8.14 Set Associative Cache

- A **hybrid** model, combining ideas from both fully associative and direct mapped caches.
- The cache is divided into a number of equal-sized **sets**.
- A given cache line maps to only *one* specific set (this part is **directly mapped**, just like direct-mapped caching, but mapping to a *set* rather than to one single position).
- Within its mapped set, however, the cache line can be placed *anywhere* available within that set (this part is **fully associative**, but only searching within the small set rather than the whole cache).
- If a set can accommodate N cache lines simultaneously, the scheme is called an **N -way set associative cache**. (A direct mapped cache is the special case $N = 1$; a fully associative cache is the special case where there is only 1 set containing every cache line, i.e. $N = \text{total number of cache entries}$.)

Worked example: 2-way set associative cache. A byte-addressable, 2-way set associative cache, for a 1024-byte main memory, with 2 bytes per cache line and total cache volume of 8 bytes.

- Since it is 2-way, each set holds 2 cache lines, so the **set size** = 2×2 bytes = 4 bytes.
- Since the total cache volume is 8 bytes, there are $8/4 = 2$ sets available.
- Address breakdown: an **Offset** field (to pick the byte within a line), a **Set Index** field (to pick which of the 2 sets a block maps to), and a **Tag** field (to identify which specific block, among those that map to the same set, currently occupies a given cache entry).
- Layout of a cache entry: Tag | Set Index | Offset | Data | Valid Bit.

Illustrative walk-through (same style as would appear in an exam): suppose the CPU successively references addresses 1, then 8, then 4 (or 5).

- Address 1 maps to Set 0; since the cache starts empty, it is placed in any free position within Set 0.
- Address 8 also maps to Set 0 (a different tag); since Set 0 can still accommodate a second cache line (2-way), it is placed alongside the entry for address 1 – *no eviction is needed*, unlike what would happen in a direct-mapped cache with only one slot per position.
- If the CPU then references address 4 or 5 (which also maps to Set 0), Set 0 is now full, so **one** of the two existing entries in Set 0 must be evicted to make room – *which* one is evicted is decided by the cache’s **replacement policy**.

This illustrates the key benefit of set associativity over direct mapping: by allowing a small amount of flexibility (multiple ways per set), far fewer unnecessary evictions occur for blocks that happen to share the same index, at a smaller search cost than a fully associative cache (only the lines within one set need to be compared, not the whole cache).

8.15 Cache Replacement Policies

Whenever a set-associative (or fully associative) cache is full and a new block must be brought in, a **replacement policy** decides which existing cache entry to evict. This decision should be made in a way that is well-aligned with the principle of locality being exploited. Common policies include:

- **Least Recently Used (LRU)**: evicts the cache entry that has gone **unused for the longest time**. Directly exploits temporal locality (the assumption that recently used data is likely to be used again soon, so the *least*-recently-used entry is the safest to discard).
- **First In First Out (FIFO)**: evicts the cache entry that has had the **longest period of stay** in the cache, regardless of how recently it was actually used.
- **Least Frequently Used (LFU)**: evicts the cache line with the **lowest access frequency** (fewest total accesses).
- **Random**: evicts a cache entry chosen at random, requiring no extra bookkeeping/state at all.

Advantages and disadvantages (summary):

- **LRU**: closely tracks real usage patterns and generally gives strong performance, but requires extra hardware/bookkeeping (tracking recency for every entry), adding cost and complexity.
- **FIFO**: very simple to implement (just a queue), but can perform poorly since it ignores whether an entry has actually been used recently – it may evict a frequently-used entry simply because it was loaded first.
- **LFU**: can effectively identify genuinely “hot” (popular) data over a long period, but requires maintaining and updating access-frequency counters for every entry, and can be slow to adapt when access patterns change suddenly.
- **Random**: extremely cheap to implement (no bookkeeping at all), and avoids certain pathological worst-case access patterns that can defeat deterministic policies, but does not exploit locality at all, so on average it usually performs worse than LRU.